

Оглавление

Введение.....	6
1 Постановка задачи и обзор аналогичных решений	7
1.1 Обзор аналогичных решений.....	7
1.1.1 Платформа Quizlet.....	7
1.1.2 Образовательная платформа Stepik.....	7
1.2 Постановка задачи	8
1.3 Выводы по разделу	9
2 Проектирование web-приложения	10
2.1 Описание структуры приложения	10
2.2 Описание базы данных	11
2.3 Развертывание сервиса	17
2.4 Роли пользователей и их функции	18
2.4 Выводы по разделу	22
3 Реализация web-приложения	24
3.1 Microsoft SQL Server.....	24
3.2 Программные библиотеки.....	27
3.3 Структура серверной части.....	29
3.3.1 Описание директорий.....	29
3.3.2 Описание маршрутов.....	30
3.4 Реализация функций для гостя	32
3.4.1 Регистрация	32
3.4.2 Аутентификация.....	33
3.4.3 Просмотр публичной информации	33
3.5 Реализация функций для обучающегося	34
3.5.1 Просмотр образовательных материалов.....	34
3.5.2 Прохождение сгенерированных тестов	34
3.5.3 Экспорт учебных материалов	35
3.6 Реализация функций для преподавателя	36
3.6.1 Загрузка исходных материалов	36
3.6.2 Настройка параметров генерации	37
3.6.3 Редактирование сгенерированного контента	38
3.6.4 Публикация учебных материалов	38
3.7 Реализация функция для администратора.....	39
3.7.1 Управление учетными записями	39
3.7.2 Конфигурация параметров AI-моделей.....	40
3.8 Реализация промежуточных обработчиков.....	40
3.8.1 Аутентификация и проверка роли.....	40
3.8.2 Загрузка исходных материалов	42
3.9 Структура клиентской части.....	43
3.9 Выводы по разделу	45
4 Тестирование web-приложения	47
4.1 Выводы по разделу	60
5 Руководство пользователя	61

5.1 Руководство гостя	61
5.1.1 Регистрация	61
5.1.2 Просмотр публичной информации	62
5.1.3 Аутентификация.....	63
5.2 Руководство обучающегося	63
5.2.1 Аутентификация.....	63
5.2.2 Прохождение сгенерированных тестов	64
5.2.3 Экспорт учебных материалов	65
5.3 Руководство преподавателя	65
5.3.1 Аутентификация.....	65
5.3.2 Настройка параметров генерации и загрузка исходных материалов	66
5.3.3 Публикация учебных материалов	67
5.4 Руководство администратора	67
5.4.1 Аутентификация.....	67
5.4.2 Конфигурация параметров AI-моделей.....	68
5.4.3 Управление учетными записями пользователей	68
5.4.4 Управление профилем	69
5.5 Выводы по разделу	69
Заключение	71
Список используемых источников.....	72
Приложение А	73
Приложение Б.....	74
Приложение В	75
Приложение Г.....	76
Приложение Д	77
Приложение Е.....	79

Введение

Современные образовательные платформы требуют инструментов, которые позволяют быстро создавать, структурировать и распространять учебный контент, а также обеспечивать удобный доступ к материалам для разных категорий пользователей. В рамках данного курсового проекта разработано SPA web-приложение интеллектуальной генерации учебного контента, предназначенное для автоматизации подготовки учебных материалов на основе загружаемых документов с использованием технологий искусственного интеллекта.

Цель курсового проекта – разработать и реализовать web-приложение для генерации и управления учебными материалами, обеспечивающее ролевой доступ (гость, студент, преподаватель, администратор), централизованное администрирование параметров ИИ, публикацию материалов и безопасное развертывание в контейнерной среде с поддержкой HTTPS.

Целевая аудитория приложения включает в себя авторов учебного контента – в первую очередь сотрудников, которые готовят лекции и практикумы по дисциплинам: загрузка исходных документов, автоматическая генерация и последующая доработка конспектов, терминов, тестов и ситуационных заданий, решение о публикации материала для аудитории, а также обучающихся – студентов, которым нужен структурированный материал для самостоятельной работы: чтение конспекта, закрепление терминов, самопроверка и прохождение тестов с фиксацией прогресса.

Для достижения поставленной цели были сформулированы следующие задачи:

1. Спроектировать архитектуру клиент–серверного приложения с разделением представления, бизнес-логики и хранилища данных (рассмотрено в разделе 2).
2. Реализовать проксирование, защищенное HTTPS-соединение и контейнеризацию (Nginx, Docker, Docker Compose) для переносимого развертывания приложения (рассмотрено в разделе 2).
3. Реализовать серверную часть и клиентскую часть (рассмотрено в разделе 3).
4. Выполнить тестирование и отладку ключевых сценариев работы системы (рассмотрено в разделе 4).

В качестве основной платформы выбрана Node.js, как среда для реализации серверной части с асинхронной событийно-ориентированной моделью. Клиентская часть реализована на JavaScript в формате SPA, взаимодействие между клиентом и сервером организовано через REST API. В качестве СУБД использована Microsoft SQL Server. Для управления статическими ресурсами, маршрутизацией и безопасным доступом применен Nginx с поддержкой HTTP/HTTPS. Развертывание системы выполнено с использованием Docker/Docker Compose, что обеспечивает переносимость и повторяемость запуска на различных платформах.

1 Постановка задачи и обзор аналогичных решений

1.1 Обзор аналогичных решений

В рамках обзора аналогичных решений проводится сравнительный анализ существующих образовательных web-платформ и сервисов генерации учебного контента на основе искусственного интеллекта. Это позволяет определить ключевые функциональные и технические требования для систем данного класса. Изучение аналогов помогает выбрать оптимальные подходы к проектированию пользовательского интерфейса, ролевой модели доступа, архитектуры REST API и структуры базы данных, необходимых для разработки конечного программного продукта – системы интеллектуальной генерации и управления учебными материалами.

1.1.1 Платформа Quizlet

Quizlet [1] – популярная образовательная веб-платформа с наборами карточек (термин-определение), режимами заучивания и проверки знаний.

Интерфейс платформы представлен на рисунке 1.1.

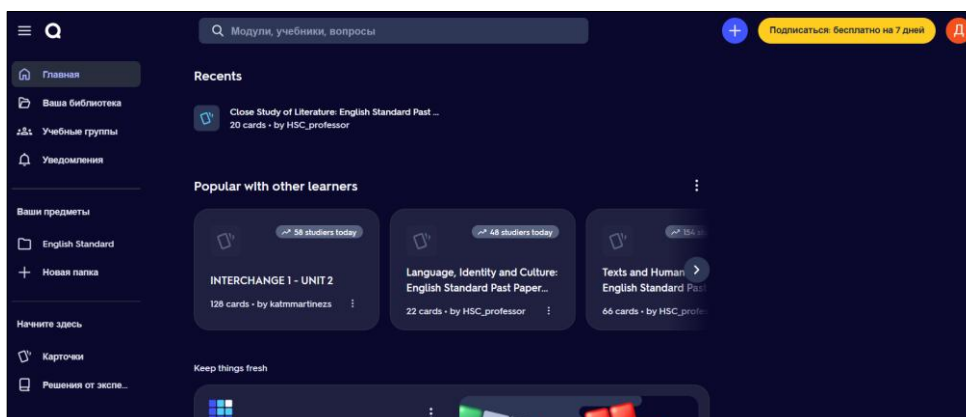


Рисунок 1.1 – Интерфейс Quizlet

Преимуществами данной платформы являются сильная визуализация карточек и тренировок, большая база готовых наборов, понятная навигация для студента.

К недостаткам следует отнести то, что платформа ориентирована прежде всего на готовые наборы карточек и типовые режимы повторения, а не на полный цикл создания учебного модуля в рамках одной организации. Кроме того, условия использования сервиса зависят от тарифной политики и могут ограничивать возможности бесплатного использования.

1.1.2 Образовательная платформа Stepik

Stepik [2] – открытая образовательная платформа: курсы, уроки, автоматические задания и тесты, роли преподавателя и обучающегося,

публикация материалов. Основной акцент Stepik сделан на готовых курсах и их проведении, а не на автоматической генерации учебного контента из произвольного загружаемого файла.

Интерфейс платформы представлен на рисунке 1.2.

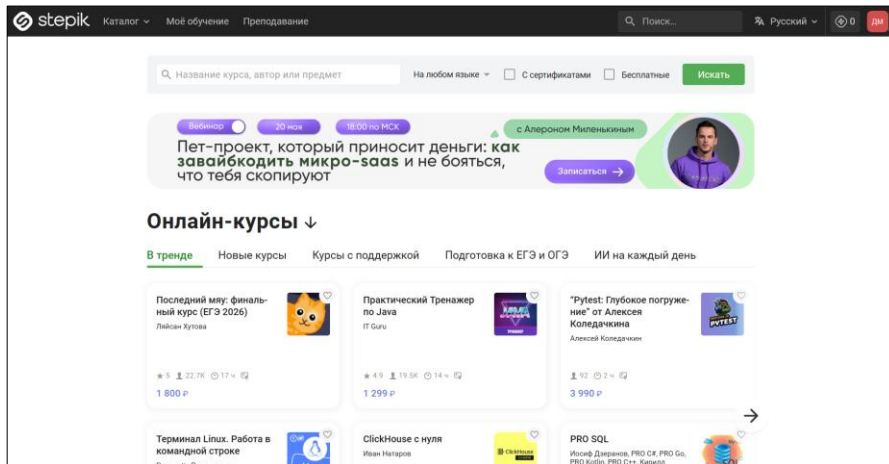


Рисунок 1.2 – Интерфейс Stepik

Преимуществами данной платформы являются зрелая модель курса, уроков и тестирования, а также понятная логика для студента и автора курса. Сайт демонстрирует наглядный пример организации учебного контента в веб-среде.

Основные недостатки платформы Stepik включают неровное качество контента из-за большого числа бесплатных курсов, отсутствие качественной обратной связи, технические сбои и слабый фокус на прикладных проектах. Платформа лучше подходит для введения в тему, чем для глубокого профессионального обучения.

1.2 Постановка задачи

На сегодняшний день актуальность разработки удобной и функциональной веб-платформы для поддержки учебного процесса с использованием технологий искусственного интеллекта высока. Автоматизация подготовки и сопровождения образовательных материалов снижает нагрузку на преподавателя, ускоряет выпуск структурированного контента и повышает доступность обучения, в то время как многие существующие решения либо не интегрируют генерацию контента в единый сценарий, либо жёстко привязаны к внешним сервисам без возможности развёртывания на инфраструктуре учебного заведения и настройки моделей под внутренние регламенты.

Целевой аудиторией приложения являются обучающиеся и преподаватели образовательных организаций.

Обзор аналогов позволяет проанализировать преимущества и недостатки альтернативных подходов и сформулировать требования к разрабатываемому в рамках курсового проекта программному средству.

Функционально веб-приложение должно поддерживать роли «Гость», «Обучающийся», «Преподаватель», «Администратор» и реализовывать следующие функции:

- регистрация;
- аутентификация;
- просмотр публичной информации;
- просмотр образовательных материалов;
- прохождение сгенерированных тестов;
- экспорт учебных материалов;
- загрузка исходных материалов;
- настройка параметров генерации;
- редактирование сгенерированного контента;
- публикация учебных материалов;
- управление учетными записями;
- конфигурация параметров AI-моделей.

Диаграмма вариантов использования представлена в приложении А.

1.3 Выводы по разделу

Проведённый анализ предметной области показал, что поддержка учебного процесса с применением генеративных моделей требует единого веб-приложения с явным разграничением прав доступа и сквозным сценарием работы с материалами. Разделение ролей на гостя, обучающегося, преподавателя и администратора позволяет охватить полный цикл: от ограниченного доступа к публичной информации и регистрации до просмотра и проверки знаний по опубликованным материалам, загрузки исходных документов с настройкой генерации, редактирования и публикации контента, а также централизованного управления учётными записями и параметрами AI-моделей.

Сравнительный обзор существующих образовательных и вспомогательных сервисов (в том числе ориентированных на курсы, карточки или работу с документами) подтверждает наличие отдельных полезных функций, однако выявляет типичные ограничения для задач курсового проекта.

Таким образом, требования к разрабатываемой системе сформулированы с учётом как целей технического задания, так и пробелов у аналогов: акцент делается на автономности развёртывания, единой предметной модели данных и управляемой генерации контента, а не на фрагментарном наборе внешних сервисов. Это предопределяет необходимость проектирования защищённого REST-интерфейса, надёжного хранения материалов и настроек в СУБД, а также явной политики доступа по ролям на всех уровнях приложения.

2 Проектирование web-приложения

2.1 Описание структуры приложения

Для реализации платформы поддержки учебного процесса с генерацией образовательного контента на базе больших языковых моделей необходимо спроектировать наглядный пользовательский интерфейс, обеспечивающий эффективную работу с материалами, тестами и административными настройками при обращении к серверу приложения и системе хранения данных на базе Microsoft SQL Server. Программное решение должно поддерживать асинхронную обработку HTTP-запросов на стороне сервера: выдача опубликованных материалов и тестов, приём загрузок исходных файлов (в том числе PDF и DOCX), вызов внешнего API генерации, сохранение черновиков и опубликованного контента, фиксация прогресса обучающихся. Рациональная организация запросов к снижает задержки при работе с каталогом материалов, результатами тестов и служебными сущностями.

В состав функциональных возможностей веб-приложения должны входить следующие инструменты:

- ролевая модель с разграничением прав для гостя, обучающегося, преподавателя и администратора;
- регистрация и аутентификация пользователей;
- для гостя – просмотр публичной информации;
- для обучающегося – просмотр образовательных материалов, прохождение сгенерированных тестов, экспорт учебных материалов, учёт прогресса;
- для преподавателя – загрузка исходных материалов, настройка параметров генерации, редактирование сгенерированного контента и его публикация;
- для администратора – управление учётными записями и конфигурация параметров AI-моделей (включая обращение к провайдеру, например Groq).

Серверная часть (backend) реализована на платформе Node.js [3] с использованием веб-фреймворка Express.js [4], обрабатывающего запросы REST API. Для работы с Microsoft SQL Server применяется драйвер mssql: доступ к данным организуется через SQL-запросы и пул соединений без отдельного ORM-слоя, что соответствует текущей реализации проекта и упрощает контроль производительности критичных операций.

Клиентская часть (frontend) построена как одностраничное приложение (SPA) на библиотеке React [5] с использованием среды сборки Vite [6]. Маршрутизация реализована средствами React Router [7]. Взаимодействие с backend выполняется по HTTP (в том числе через Axios), отображение уведомлений пользователю – с применением подходящих клиентских компонентов и библиотек интерфейса, согласованных с выбранной в проекте версткой.

Nginx в составе развёртывания выполняет роль обратного прокси и точки входа для HTTP/HTTPS, обеспечивая раздачу статики фронтенда (после сборки) и проксирование запросов к API Node.js.

Для воспроизводимости среды и согласованности работы на машине разработчика и на сервере применяется контейнеризация Docker: компоненты стека (приложение Node.js, Microsoft SQL Server, Nginx) изолируются в контейнерах. Docker Compose используется для описания и совместного запуска сервисов, автоматизации сборки и старта проекта в единой конфигурации.

2.2 Описание базы данных

Реляционная база данных в рамках курсового проекта выступает основным хранилищем структурированных данных образовательной платформы AI-Education: она обеспечивает ссылочную целостность, однозначность связей между сущностями и поддержку бизнес-логики веб-приложения (роли, учебные материалы, прогресс обучения, настройки генерации контента). Архитектура спроектирована с учётом ролевой модели доступа: справочник ролей жёстко связан с учётными записями пользователей, что позволяет разграничивать права гостя, обучающегося, преподавателя и администратора на уровне данных и API. Ядро хранилища ориентировано на каталог учебных материалов (текст конспекта, термины, тесты, задания), привязку к тематическим категориям, публикацию и учёт индивидуального прогресса по каждому материалу.

Доступ к базе из серверной части реализован через параметризованные SQL-запросы и асинхронный драйвер mssql, что снижает риск SQL-инъекций и даёт предсказуемое выполнение операций чтения и изменения данных при работе REST API и формировании представлений в пользовательском интерфейсе. Отдельная таблица системных параметров выделена для хранения конфигурации, не требующей реляционных связей с пользовательскими сущностями (например, ключи и параметры внешних сервисов).

Диаграмма базы данных представлена в приложении Б.

На этапе разработки концептуальной модели были формализованы бизнес-требования и основные процессы веб-приложения AI-Education. К основным сущностям, отражённым в логической структуре из шести таблиц, относятся:

- роли – фиксированный перечень типов участников системы (обучающийся, преподаватель, администратор) для разграничения прав доступа к функциям каталога, создания и модерации материалов, администрирования пользователей;
- пользователи – зарегистрированные участники с хэшем пароля, реквизитами ФИО, признаком блокировки, привязкой к роли;
- категории – тематические рубрики учебного контента, обеспечивающие классификацию и фильтрацию материалов в интерфейсе;

– учебные материалы – единица контента, создаваемая преподавателем: заголовок, категория, структурированные поля конспекта, терминов, тестов и заданий, признаки публикации и публичности, временные метки создания и обновления;

– прогресс по материалу – единица контента с атрибутами освоения (прочтение конспекта, термины, прохождение теста), счётчиками попыток и накопленной статистикой по баллам;

– конфигурация системы – параметры приложения и интеграций без обязательных внешних ключей к остальным таблицам.

Таблица Roles хранит в себе информацию о доступных ролях пользователей в системе. Структура Roles представлена в таблице 2.1.

Таблица 2.1 – Структура таблицы Roles

Атрибут	Тип данных	Ограничение	Назначение
Id	INT	PRIMARY KEY, IDENTITY	Уникальный идентификатор роли
RoleName	NVARCHAR(50)	NOT NULL, UNIQUE	Наименование роли

Таблица Users хранит учетные данные пользователей приложения, включая привязку к роли и время регистрации. Структура Users представлена в таблице 2.2.

Таблица 2.2 – Структура таблицы Users

Атрибут	Тип данных	Ограничение	Назначение
Id	INT	PRIMARY KEY, IDENTITY	Уникальный идентификатор пользователя
Username	NVARCHAR(100)	NOT NULL, UNIQUE	Логин (email) пользователя
PasswordHash	NVARCHAR(255)	NOT NULL	Хэш пароля
RoleId	INT	NOT NULL, FOREIGN KEY	Идентификатор роли пользователя
FirstName	NVARCHAR(100)	Допускается NULL	Имя пользователя
LastName	NVARCHAR(100)	Допускается NULL	Фамилия пользователя
MiddleName	NVARCHAR(100)	Допускается NULL	Отчество пользователя
IsBlocked	BIT	DEFAULT 0	Признак блокировки пользователя

Продолжение таблицы 2.2

Атрибут	Тип данных	Ограничение	Назначение
CreatedAt	DATETIME	DEFAULT GETUTCDATE	Дата регистрации учетной записи

Таблица Categories хранит перечень тематических категорий учебных материалов и используется для классификации и фильтрации контента в каталоге. Каждое наименование категории уникально. Структура Categories представлена в таблице 2.3.

Таблица 2.3 – Структура таблицы Categories

Атрибут	Тип данных	Ограничение	Назначение
Id	INT	PRIMARY KEY, IDENTITY	Уникальный идентификатор категории
CategoryName	NVARCHAR(50)	NOT NULL, UNIQUE	Наименование категории

Таблица EducationalMaterials хранит учебные материалы, создаваемые преподавателями: заголовок, привязку к автору и категории, имя исходного файла документа, структурированное содержание (конспект, термины, тесты, самопроверка, практическое задание) в текстовом виде, признаки публикации и доступности, а также даты создания и последнего обновления записи. Структура EducationalMaterials представлена в таблице 2.4.

Таблица 2.4 – Структура таблицы EducationalMaterials

Атрибут	Тип данных	Ограничение	Назначение
Id	INT	PRIMARY KEY, IDENTITY	Уникальный идентификатор материала
TeacherId	INT	NOT NULL, FOREIGN KEY	Преподаватель, создавший материал
Title	NVARCHAR(255)	NOT NULL	Название материала
CategoryId	INT	NOT NULL, FOREIGN KEY	Тематическая категория
OriginalFileName	NVARCHAR(255)	Допускается NULL	Имя загруженного исходного файла (PDF/DOCX)
Summary	NVARCHAR(MAX)	Допускается NULL	Текст конспекта

Продолжение таблицы 2.4

Атрибут	Тип данных	Ограничение	Назначение
Terms	NVARCHAR(MAX)	Допускается NULL	Термины и определения
Quizzes	NVARCHAR(MAX)	Допускается NULL	Вопросы теста и варианты ответов
SelfCheck	NVARCHAR(MAX)	Допускается NULL	Вопросы для самопроверки
PracticalTask	NVARCHAR(MAX)	Допускается NULL	Ситуационная задача
IsPublished	BIT	DEFAULT 0	Признак публикации в каталоге для обучающихся
IsPublic	BIT	DEFAULT 0	Признак доступности для гостей
CreatedAt	DATETIME	DEFAULT GETUTCDATE()	Дата создания материала
UpdatedAt	DATETIME	DEFAULT GETUTCDATE()	Дата последнего изменения материала

Таблица StudentMaterialProgress фиксирует индивидуальный прогресс обучающегося по каждому учебному материалу: флаги освоения этапов (конспект, термины, тест), результаты последней попытки теста, накопленную статистику по попыткам и баллам, время последнего открытия материала и время обновления записи. Для одной пары «студент-материал» допускается не более одной строки (уникальное ограничение по StudentId и MaterialId). Структура StudentMaterialProgress представлена в таблице 2.5.

Таблица 2.5 – Структура таблицы StudentMaterialProgress

Атрибут	Тип данных	Ограничение	Назначение
Id	INT	PRIMARY KEY, IDENTITY	Уникальный идентификатор записи прогресса
StudentId	INT	NOT NULL, FOREIGN KEY	Обучающийся
MaterialId	INT	NOT NULL, FOREIGN KEY	Учебный материал
SummaryRead	BIT	NOT NULL, DEFAULT 0	Признак прочтения конспекта

Продолжение таблицы 2.5

Атрибут	Тип данных	Ограничение	Назначение
TermsLearned	BIT	NOT NULL, DEFAULT 0	Признак освоения блока терминов
QuizCompleted	BIT	NOT NULL, DEFAULT 0	Признак прохождения теста
LastQuizScore	INT	Допускается NULL	Процент последней попытки теста
QuizAttempts	INT	NOT NULL, DEFAULT 0	Число попыток прохождения теста
TotalQuizPercent	INT	NOT NULL, DEFAULT 0	Накопленный показатель по результатам тестов
LastOpenedAt	DATETIME	Допускается NULL	Время последнего открытия материала
UpdatedAt	DATETIME	NOT NULL, DEFAULT GETUTCDATE()	Время последнего изменения записи прогресса
—	—	UNIQUE (StudentId, MaterialId)	Одна запись прогресса на пару студент — материал

Таблица SystemConfigs предназначена для хранения пар «ключ-значение» глобальных параметров приложения: настройки доступа к внешним сервисам (в частности, ключи и параметры для работы с ИИ), иные конфигурационные константы, которые могут изменяться администратором без правки исходного кода. Таблица не содержит внешних ключей к пользователям и учебному контенту: это автономный справочник настроек.

Таблица 2.6 – Структура таблицы SystemConfigs

Атрибут	Тип данных	Ограничение	Назначение
ConfigKey	NVARCHAR(50)	PRIMARY KEY	Идентификатор настройки
ConfigValue	NVARCHAR(MAX)	Допускается NULL	Значение параметра

Между таблицами реализованы связи типа «один ко многим»: каждая запись в родительской таблице может соответствовать нескольким записям в дочерней, а в дочерней таблице хранится внешний ключ, ссылающийся на первичный ключ родительской. Отдельная таблица SystemConfigs внешними ключами ни с кем не связана и служит для хранения пар «ключ-значение» параметров системы.

Логическая связь «многие ко многим» между множеством обучающихся и множеством материалов в физической модели представлена таблицей StudentMaterialProgress: к одному пользователю с ролью обучающегося относится много строк прогресса, к одному материалу – тоже много строк (от разных студентов), при этом пара (StudentId, MaterialId) уникальна.

В таблице 2.7 приведены все связи «один ко многим», задаваемые внешними ключами в базе данных приложения AI-Education.

Таблица 2.7 – Связи «один ко многим» в базе данных

Главная таблица	Зависимая таблица	Внешний ключ	Описание связи
Roles	Users	RoleId	У одной роли много пользователей, а у каждого пользователя одна роль
Categories	EducationalMaterials	CategoryId	У одной категории много материалов, а каждый материал относится к одной категории
Users	EducationalMaterials	TeacherId	У одного преподавателя много созданных материалов, а каждый материал имеет одного автора
Users	StudentMaterialProgress	StudentId	У одного обучающегося много записей прогресса по разным материалам, а каждая строка прогресса относится к одному студенту

Продолжение таблицы 2.7

Главная таблица	Зависимая таблица	Внешний ключ	Описание связи
EducationalMaterials	StudentMaterialProgress	MaterialId	У одного материала много записей прогресса, а каждая строка относится к одному материалу

Описанные связи между таблицами превращают набор отдельных хранилищ в единую реляционную структуру: дублирование сведений сводится к минимуму, а общие справочники и сущности переиспользуются через ссылки. Механизм ссылочной целостности обеспечивает согласованность базы на уровне СУБД и препятствует появлению записей, не согласованных с уже существующими данными.

2.3 Развертывание сервиса

Для понимания структуры разрабатываемого приложения и как его разворачивать, необходимо построить диаграмму развертывания. Диаграмма развертывания web-приложения представлена на рисунке 2.2.

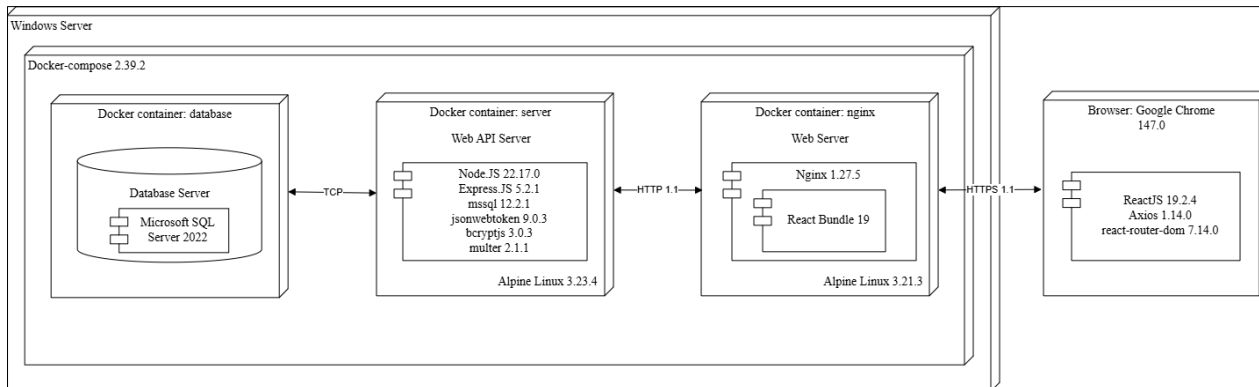


Рисунок 2.2 – Диаграмма развертывания

Серверная часть разворачивается средствами Docker Compose и включает изолированные контейнеры. Хранение данных обеспечивает контейнер с Microsoft SQL Server 2022 [8]. Прикладной сервер (Node.js, Express) обращается к СУБД по TCP с использованием драйвера mssql [9] и параметризованных SQL-запросов. Аутентификация и загрузка файлов поддерживаются пакетами jsonwebtoken, bcryptjs и multer.

Контейнер на базе Nginx выполняет роль обратного прокси и раздаёт статическую клиентскую часть, собранную Vite из исходников React. Запросы к API по префиксу /api/ проксируются на сервер приложения по HTTP во внутренней сети Docker.

На стороне клиента браузер взаимодействует с Nginx по HTTPS. Выполняемое в браузере приложение построено на React с React Router для маршрутизации и Axios для обращений к REST API.

При первом запуске может выполняться однократная инициализация схемы БД отдельным контейнером с утилитами mssql-tools.

2.4 Роли пользователей и их функции

Для корректного и безопасного взаимодействия пользователей с web-приложением необходимо не только реализовать требуемый функционал, но и чётко разграничить права доступа к данным и операциям в зависимости от того, кто обращается к системе. Это позволяет снизить риск несанкционированных действий, упростить сопровождение интерфейса (каждая категория пользователей видит только релевантные разделы) и приблизить поведение системы к принятым в образовательной практике правилам разделения обязанностей между участниками учебного процесса.

В ходе проектирования были выделены четыре роли: «Гость», «Обучающийся», «Преподаватель» и «Администратор». Каждая роль отражает типичный сценарий использования платформы – от ознакомления с открытыми материалами без регистрации до полного административного контроля над пользователями, справочниками и параметрами работы сервиса.

Перечень ролей, доступных в приложении, а также их описание и права доступа представлены в таблице 2.8.

Таблица 2.8 – Роли пользователей

Роль	Общее описание	Доступ
Гость	Пользователь без учетной записи.	Просмотр публичной информации, переход к страницам входа и регистрации.
Обучающийся	Зарегистрированный пользователь с ролью Student.	Просмотр опубликованных учебных материалов, работа с сгенерированными материалами (конспект, тесты, термины), просмотр прогресса, мини-статистика, экспорт материалов.
Преподаватель	Зарегистрированный пользователь с ролью Teacher.	Загрузка документа (PDF/DOCX), генерация материала с помощью ИИ по заданным параметрам, просмотр и редактирование своих материалов, публикация/снятие с публикации, просмотр чужих опубликованных материалов.
Администратор	Зарегистрированный пользователь с ролью Admin.	Настройка системных параметров, управление пользователями, категориями и материалами.

Роль «Гость» предназначена для пользователей, которые обращаются к системе без создания учётной записи или до момента входа в личный кабинет. Такой режим доступа типичен для первичного знакомства с платформой, просмотра открытых учебных материалов и оценки возможностей сервиса без обязательства регистрироваться. При этом объём операций у гостя намеренно ограничен: он не может изменять учебный контент, управлять пользователями или использовать функции, требующие идентификации пользователя.

Функции роли «Гость» и их описание представлены в таблице 2.9.

Таблица 2.9 – Функции роли «Гость»

Функция	Описание
Просмотр публичной информации	Доступ к главной странице приложения без авторизации: каталог материалов с признаком публичности, фильтрация по категориям и поиск по названию или ФИО автора. Открытие карточки материала с ограниченным просмотром содержимого.
Аутентификация	Переход на страницу входа, ввод логина (email) с проверкой пароля и переход в учебную панель.
Регистрация	Переход на страницу регистрации, заполнение ФИО, логина (email) и пароля по заданным правилам. Пароль хэшируется. По умолчанию назначается роль «Обучающийся».

Функции роли «Обучающийся» и их описание продемонстрированы в таблице 2.10.

Таблица 2.10 – Функции роли «Обучающийся»

Функция	Описание
Просмотр каталога образовательных материалов	Просмотр списка опубликованных учебных материалов в виде карточек (категория, автор, даты).
Поиск и фильтрация материалов	Поиск по названию темы и ФИО автора. Выбор категории из справочника.
Сортировка материалов	Упорядочивание по дате создания, дате обновления, названию.
Быстрый переход к последнему материалу	Блок «Продолжить обучение» по последним открытым материалам.
Просмотр содержимого материала	Конспект, термины, тест, самопроверка, ситуационная задача.
Прохождение сгенерированных тестов	Выбор ответов, проверка, отображение результата, сброс попытки.
Учет прогресса по материалу	Чекбоксы «конспект / термины / тест» для учета прогресса.

Продолжение таблицы 2.10

Функция	Описание
Мини-статистика	Просмотр показателей: открыто материалов, попыток тестов, средний результат по попыткам.
Экспорт материала	Сохранение текущего материала в текстовый файл.
Управление профилем	Просмотр информации о пользователе, возможность смены логина и пароля.

Роль «Преподаватель» ориентирована на сотрудников, ответственных за подготовку и сопровождение учебного контента в рамках дисциплины или курса. В отличие от обучающегося, преподаватель не ограничивается только потреблением опубликованных материалов: ему доступны сценарии загрузки исходных документов, автоматизированная генерация структурированных элементов учебного модуля (конспект, термины, тесты, задания) с последующей редакцией и методической правкой, а также решение о публикации материала для целевой аудитории. Таким образом, преподаватель выступает связующим звеном между «сырым» учебным источником и качественным цифровым представлением дисциплины в системе.

Функции роли «Преподаватель» и их описание продемонстрированы в таблице 2.11.

Таблица 2.11 – Функции роли «Преподаватель»

Функция	Описание
Настройка параметров генерации и загрузка исходных материалов	Загрузка PDF/DOCX файлов, настройка параметров генерации: выбор темы, объема конспекта и указание количества вопросов теста.
Генерация учебного материала	Генерация учебного материала на основе параметров, которые были заданы.
Просмотр образовательных материалов	Просмотр образовательных материалов своих и материалов, сгенерированных другими преподавателями.
Просмотр материала	Режим просмотра: конспект, термины, тест, ситуационная задача, самопроверка. Материалы других преподавателей доступны для экспорта и тесты доступны для прохождения.
Экспорт материала	Выгрузка содержимого в текстовый файл из режима просмотра.
Редактирование материала	Изменение заголовка, категории, конспекта, терминов, вопросов самопроверки, ситуационной задачи и вопросов к ней. Сохранение изменений.
Регенерация вопроса теста	Замена вопросов ИИ в режиме редактирования.

Продолжение таблицы 2.11

Функция	Описание
Доступ для гостей	Флаг «Сделать доступным для гостей» в режиме редактирования.
Публикация и снятие с публикации	Публикация черновика, снятие опубликованного материала с подтверждением.
Удаление материала	Удаление материала с подтверждением.
Фильтрация и поиск материалов	Поиск по названию темы и ФИО автора. Выбор категории из справочника.
Сортировка материалов	Упорядочивание по дате создания, дате обновления, названию.
Управление профилем	Просмотр информации о пользователе, возможность смены логина и пароля.

Роль «Администратор» предполагает наивысший уровень полномочий в рамках web-приложения и предназначена для сотрудников, которые обеспечивают устойчивую работу, безопасность и актуальность конфигурации платформы. Как правило, это задачи, не относящиеся к повседневной методической работе преподавателя: ведение учётных записей и ролей, поддержание справочной структуры (категорий), настройка параметров взаимодействия с внешними сервисами (в частности, с нейросетевым API), а также контроль размещаемого контента. Наличие отдельной административной роли позволяет централизовать ответственность за эксплуатацию системы и снизить вероятность ошибочных изменений со стороны пользователей, не уполномоченных на такие действия.

Функции роли «Администратор» и их описание продемонстрированы в таблице 2.12.

Таблица 2.12 – Функции роли «Администратор»

Функция	Описание
Конфигурация параметров AI-моделей	Загрузка списка доступных моделей и текущих параметров. Выбор модели, стиля, температуры и лимита токенов. Сохранение выбранных параметров.
Управление учетными записями пользователей	Просмотр списка учетных записей.
Смена роли пользователя	Назначение роли пользователю.
Блокировка и разблокировка	Блокировка и разблокировка пользователей.
Удаление пользователя	Удаление учетной записи с подтверждением.
Поиск и фильтрация пользователей	Поиск пользователя по ФИО или логину, фильтрация по роли.
Сортировка пользователей	Сортировка по имени.

Продолжение таблицы 2.12

Функция	Описание
Просмотр категорий	Просмотр добавленных категорий с числом материалов.
Добавление категории	Добавление тематической категории.
Поиск, фильтр и сортировка	Поиск по названию, фильтрация и сортировка по названию и загрузке.
Редактирование категории	Изменение названия категории.
Удаление категории	Удаление категории с подтверждением.
Просмотр материалов	Просмотр материалов, добавленных преподавателями.
Поиск, фильтр и сортировка	Поиск по названию и ФИО автора, фильтрация по категории, сортировка по датам и названию
Просмотр материала	Открытие содержимого материала в режиме просмотра (конспект, термины, тест с подсветкой правильных ответов, ситуационная задача, вопросы для самопроверки).
Снятие с публикации	Снятие опубликованных материалов с публикации
Удаление материала	Удаление материала с подтверждением.
Управление профилем	Просмотр информации о пользователе, возможность смены логина и пароля.

На основе спроектированного распределения ролей и их функционала можно сделать вывод, что в разрабатываемом веб-приложении реализована четкая иерархия прав доступа, обеспечивающая безопасность данных и удобство взаимодействия для всех категорий посетителей. Архитектура системы выстроена таким образом, что каждый уровень предоставляет строго необходимый объем возможностей.

2.4 Выводы по разделу

1. Спроектирована архитектура клиент-серверного веб-приложения AI-Education на стеке React (сборка Vite), Node.js и Express, обеспечивающая наглядный интерфейс для гостей, обучающихся, преподавателей и администратора, а также асинхронную обработку запросов к REST API и внешним сервисам генерации контента.

2. Разработана и визуализирована нормализованная реляционная модель данных под управлением Microsoft SQL Server: хранение пользователей и ролей, категорий, учебных материалов с JSON-структурированными полями, прогресса обучающихся и системных настроек. Обеспечены ссылочная целостность и уникальность пары «студент-материал» в таблице прогресса.

3. Сформирована схема развёртывания на базе Docker Compose с изолированными контейнерами SQL Server, Node.js (Express) и Nginx (раздача

статической SPA и обратный прокси к API), что поддерживает воспроизводимость окружения, разделение сервисов и безопасный канал HTTPS со стороны клиента.

4. Определена и описана ролевая модель («Гость», «Обучающийся», «Преподаватель», «Администратор») с разграничением доступа к публичному каталогу, учебной панели, генерации и редактированию материалов, а также к администрированию пользователей, категорий, материалов и параметров ИИ. Аутентификация реализована с использованием JWT и проверки учётных данных на сервере.

5. На основе выполненного проектирования можно заключить, что выбранные архитектурные, инфраструктурные и логические решения соответствуют целям курсового проекта и создают устойчивую основу для безопасной и расширяемой образовательной платформы с поддержкой ИИ-ассистированной подготовки учебных материалов.

3 Реализация web-приложения

3.1 Microsoft SQL Server

Для доступа к данным в серверной части приложения AI-Education используется нативный драйвер mssql для Node.js и пул соединений с СУБД Microsoft SQL Server. Запросы формируются в виде параметризованных шаблонных строк. Значения из запроса подставляются отдельно от текста SQL, что снижает риск SQL-инъекций и упрощает передачу данных из тела HTTP-запроса в базу. Использование явного SQL даёт прямой контроль над текстом запросов, планом выполнения на стороне СУБД и объёмом передаваемых данных, что уместно при работе с полями.

Результаты выборок обрабатываются через recordset; операции изменения данных выполняются в тех же контроллерах, что и бизнес-логика REST API (Express).

Соответствие сущностей предметной области таблицам базы данных представлено в таблице 3.1.

Таблица 3.1 – Соответствие сущностей предметной области таблицам базы данных

Сущность	Название таблицы
Роли пользователей	Roles
Тематические категории материалов	Categories
Учетные записи и профили	Users
Учебные материалы	EducationalMaterials
Прогресс обучающегося по материалу	StudentMaterialProgress
Системные параметры	SystemConfigs

Код, описывающий подключение к Microsoft SQL Server, представлен в приложении В.

Код, описывающий таблицу ролей Roles, приведен в листинге 3.1.

```
CREATE TABLE Roles (
    Id INT PRIMARY KEY IDENTITY(1,1),
    RoleName NVARCHAR(50) NOT NULL UNIQUE
);
INSERT INTO Roles (RoleName) VALUES ('Student'), ('Teacher'),
('Admin');
```

Листинг 3.1 – Таблица Roles

Таблица Roles задаёт перечень ролей в системе AI-Education: Id – суррогатный первичный ключ с автонумерацией (IDENTITY). RoleName – уникальное текстовое имя роли (обучающийся, преподаватель, администратор). Таблица выступает справочником и связана с сущностью пользователей отношением «один ко многим»: в таблице Users поле RoleId

ссылается на Roles.Id, что позволяет назначать каждому пользователю ровно одну роль и разграничивать доступ в приложении.

Связь пользователей с ролями и учётные данные описаны в таблице Users. Фрагмент скрипта приведён в листинге 3.2.

```
CREATE TABLE Users (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Username NVARCHAR(100) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    RoleId INT NOT NULL FOREIGN KEY REFERENCES Roles(Id),
    FirstName NVARCHAR(100),
    LastName NVARCHAR(100),
    MiddleName NVARCHAR(100),
    IsBlocked BIT DEFAULT 0,
    CreatedAt DATETIME DEFAULT GETUTCDATE()
);
```

Листинг 3.2 – Таблица Users

Таблица Users хранит учётные записи: Username (email) и PasswordHash (хэш пароля). RoleId – внешний ключ на Roles, обязательное поле (NOT NULL), фиксирующее роль пользователя. Поля FirstName, LastName, MiddleName задают ФИО для отображения в интерфейсе. IsBlocked – признак блокировки (используется при входе и в проверке JWT). CreatedAt – время создания записи (UTC). Связь с Roles обеспечивает иерархию прав, от этой таблицы далее отходят связи «один ко многим» к EducationalMaterials (автоматериалы) и StudentMaterialProgress (прогресс по материалам).

Структура справочника тематических категорий задаётся таблицей Categories. Фрагмент скрипта приведён в листинге 3.3.

```
CREATE TABLE Categories (
    Id INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(100) NOT NULL UNIQUE
);
```

Листинг 3.3 – Таблица Categories

Таблица Categories хранит рубрикатор учебного контента: Id – первичный ключ с IDENTITY. CategoryName – уникальное наименование категории (NOT NULL, UNIQUE). С таблицей EducationalMaterials таблица связана как «один ко многим»: у одной категории может быть много материалов, каждый материал ссылается ровно на одну категорию через внешний ключ CategoryId. Категории используются при фильтрации каталога, при генерации материалов преподавателем и при администрировании рубрик.

Содержимое учебных материалов и их метаданные хранятся в таблице EducationalMaterials. Фрагмент скрипта, который описывает эту таблицу приведён в листинге 3.4.

```

CREATE TABLE EducationalMaterials (
    Id INT PRIMARY KEY IDENTITY(1,1),
    TeacherId INT NOT NULL FOREIGN KEY REFERENCES Users(Id),
    Title NVARCHAR(255) NOT NULL,
    CategoryId INT NOT NULL FOREIGN KEY REFERENCES
Categories(Id),
    OriginalFileName NVARCHAR(255),
    Summary NVARCHAR(MAX),
    Terms NVARCHAR(MAX),
    Quizzes NVARCHAR(MAX),
    SelfCheck NVARCHAR(MAX),
    PracticalTask NVARCHAR(MAX),
    IsPublished BIT DEFAULT 0,
    IsPublic BIT DEFAULT 0,
    CreatedAt DATETIME DEFAULT GETUTCDATE(),
    UpdatedAt DATETIME DEFAULT GETUTCDATE()
);

```

Листинг 3.4 – Таблица EducationalMaterials

Таблица EducationalMaterials описывает единицу учебного контента. TeacherId – автор (преподаватель), обязательная связь с Users. CategoryId – обязательная связь с Categories. Title – тема материала. Поля Summary, Terms, Quizzes, SelfCheck, PracticalTask имеют тип NVARCHAR(MAX) и в приложении содержат структурированные данные (в том числе в текстовом виде JSON). OriginalFileName фиксирует имя загруженного исходного файла. IsPublished и IsPublic задают видимость в каталоге обучающихся и доступность гостям на публичной странице. CreatedAt и UpdatedAt – метки времени создания и обновления. От таблицы «один ко многим» зависят записи StudentMaterialProgress (MaterialId).

Индивидуальный прогресс обучающегося по материалу хранится в таблице StudentMaterialProgress. Фрагмент скрипта приведён в листинге 3.5.

```

CREATE TABLE StudentMaterialProgress (
    Id INT PRIMARY KEY IDENTITY(1,1),
    StudentId INT NOT NULL FOREIGN KEY REFERENCES Users(Id),
    MaterialId INT NOT NULL FOREIGN KEY REFERENCES
EducationalMaterials(Id),
    SummaryRead BIT NOT NULL DEFAULT 0,
    TermsLearned BIT NOT NULL DEFAULT 0,
    QuizCompleted BIT NOT NULL DEFAULT 0,
    LastQuizScore INT NULL,
    QuizAttempts INT NOT NULL DEFAULT 0,
    TotalQuizPercent INT NOT NULL DEFAULT 0,
    LastOpenedAt DATETIME NULL,
    UpdatedAt DATETIME NOT NULL DEFAULT GETUTCDATE(),
    CONSTRAINT UQ_StudentMaterialProgress UNIQUE (StudentId,
MaterialId));

```

Листинг 3.5 – Таблица StudentMaterialProgress

Таблица `StudentMaterialProgress` реализует связь «обучающийся-материал» с атрибутами прогресса: флаги `SummaryRead`, `TermsLearned`, `QuizCompleted`. `LastQuizScore` – результат последней попытки теста. `QuizAttempts` и `TotalQuizPercent` поддерживают учёт попыток и агрегированную статистику для интерфейса. `LastOpenedAt` – время последнего открытия материала. `UpdatedAt` – время изменения записи. Внешние ключи `StudentId` и `MaterialId` связывают строку с `Users` и `EducationalMaterials`. Ограничение `UNIQUE (StudentId, MaterialId)` гарантирует не более одной строки прогресса на пару «студент-материал». На уровне предметной области это развёрнутая форма связи «многие ко многим» между множеством студентов и множеством материалов.

Глобальные параметры приложения (в том числе настройки обращения к ИИ) хранятся в таблице `SystemConfigs`. Фрагмент скрипта приведён в листинге 3.6.

```
CREATE TABLE SystemConfigs (
    ConfigKey NVARCHAR(50) PRIMARY KEY,
    ConfigValue NVARCHAR(MAX)
);
```

Листинг 3.6 – Таблица `SystemConfigs`

Таблица `SystemConfigs` реализует хранение пар «ключ-значение» без привязки к пользовательским сущностям: `ConfigKey` – первичный ключ (`NVARCHAR(50)`). `ConfigValue` – произвольное текстовое значение (`NVARCHAR(MAX)`), допускается `NULL`. Таблица используется для конфигурации AI-модели, системного промпта, лимитов токенов и иных параметров, редактируемых администратором. Внешние ключи к остальным таблицам не объявлены, что соответствует роли автономного справочника настроек.

3.2 Программные библиотеки

Для реализации серверной части web-приложения `AI-Education` на платформе `Node.js` использован набор программных модулей, обеспечивающих работу HTTP API, доступ к `Microsoft SQL Server`, аутентификацию, загрузку файлов и разбор документов при генерации учебных материалов. Перечень основных библиотек приведён в таблице 3.2.

Таблица 3.2 – Программные библиотеки серверной части

Библиотека	Версия	Назначение
express	v5.2.1	Фреймворк для организации HTTP-сервера, маршрутизация запросов и формирования REST API.

Продолжение таблицы 3.2

Библиотека	Версия	Назначение
mssql	v12.2.1	Драйвер и клиент для подключения к Microsoft SQL Server. Выполнение параметризованных SQL-запросов без ORM.
bcryptjs	v3.0.3	Хеширование паролей и проверка хеша при входе.
jsonwebtoken	v9.0.3	Формирование и проверка JWT для stateless-аутентификации пользователей.
multer	v2.1.1	Прием multipart/form-data при загрузке PDF/DOCX для генерации материалов.
cors	v2.8.6	Настройка политика CORS для запросов к API из браузера.
dotenv	v17.3.1	Загрузка конфигурации из переменных окружения.
mammoth	v1.12.0	Извлечение текста из документов DOCX на стороне сервера.
pdf-extraction	v1.0.2	Извлечение текста из PDG для последующей обработки и генерации контента.

Клиентская часть построена как одностраничное приложение (SPA) на React. Для маршрутизации, обращений к API и вспомогательного UI подключены дополнительные модули. Список основных библиотек клиентской части представлен в таблице 3.3.

Таблица 3.3 – Программные библиотеки клиентской части

Библиотека	Версия	Назначение
react	v19.2.4	Библиотека для построения пользовательского интерфейса на основе компонентов.
react-dom	v19.2.4	Связка React с DOM браузера (монтирование и обновление дерева интерфейса).
react-router-dom	v7.14.0	Маршрутизация на стороне клиента без полной перезагрузки страницы.
axios	v1.14.0	Выполнение асинхронных HTTP-запросов к REST API.

Продолжение таблицы 3.3

Библиотека	Версия	Назначение
jwt-decode	v4.0.0	Декодирование полезной нагрузки JWT на клиенте без верификации подписи.
react-hot-toast	v2.6.0	Всплывающие уведомления об успешных операциях и ошибках.
react-icons	v5.6.0	Набор пиктограмм для оформления навигации и кнопок.
vite	v8.0.1	Сборка и dev-сервер для фронтенда. Production-сборка статических ресурсов для Nginx.
@vites/plugin-react	v6.0.1	Плагин Vite для поддержки JSX и Fast Refresh при разработке.

Использование перечисленных решений сокращает объём «инфраструктурного» кода и повышает надёжность типовых операций. В частности, связка Express и jsonwebtoken [10] задаёт основу защищённого API, а модуль mssql обеспечивает прямой доступ к Microsoft SQL Server с явным SQL-текстом и параметрами. На клиенте React и react-router-dom формируют отзывчивый интерфейс, а axios унифицирует обмен данными с сервером.

3.3 Структура серверной части

3.3.1 Описание директорий

Серверная часть приложения организована по модульному принципу и опирается на следующие группы компонентов:

- маршруты (routes) – объявляют конечные точки REST API и сопоставляют HTTP-методы и пути обработчикам в контроллерах;
- контроллеры (controllers) – принимают запросы клиента, вызывают необходимую бизнес-логику, выполняют обращения к базе данных и формируют ответ;
- сервисы (services) – выносят интеграцию с внешним ИИ и вспомогательную логику генерации контента, чтобы не перегружать контроллеры;
- конфигурация (config) – параметры подключения к Microsoft SQL Server и экспорт объекта доступа к СУБД для всего приложения;
- промежуточное ПО (middlewares) – проверка JWT, разграничение ролей, обработка multipart-загрузки файлов документов.

Структура данных задаётся таблицами в Microsoft SQL Server, а запросы выполняются напрямую через драйвер mssql в контроллерах. В таблице 3.4 приведён список директорий серверной части проекта разработки web-приложения и их назначение.

Таблица 3.4 – Директории серверной части

Директория	Назначение
config	Настройка подключения к Microsoft SQL Server: параметры пула, экспорт mssql для параметризованных запросов из контроллеров.
controllers	Обработчики запросов: аутентификация и профиль, учебные материалы, публичный каталог, администрирование пользователей, категорий и конфигурации.
middlewares	Проверка JWT и ролей. Загрузка файлов PDF/DOCX для генерации.
routes	Маршрутизаторы Express: подключение путей API к методам контроллеров и цепочкам middleware.
services	Сервис обращения к ИИ и сопутствующая логика: генерация и доработка фрагментов учебного контента с учётом настроек из БД.

Точка входа процесса – файл `server.js` в корне каталога `backend`: создание приложения Express, подключение CORS, маршрутов, запуск прослушивания порта и инициализация соединения с БД.

3.3.2 Описание маршрутов

На сервере используется маршрутизация Express: каждому сочетанию HTTP-метода и URL соответствует функция-обработчик из модуля контроллера. Часть маршрутов дополнительно проходит через middleware (проверка JWT, проверка роли, `multer` для загрузки файла).

Таблица соответствия маршрутов контроллерам и функциям в исходном коде представлена в таблице 3.5.

Таблица 3.5 – Соответствие маршрутов контроллерам и функциям

Метод	Маршрут	Контроллер	Метод контроллера
POST	/api/auth/register	authController	register
POST	/api/auth/login	authController	login
PUT	/api/auth/update-profile	authController	updateProfile
GET	/api/auth/profile	authController	getProfile
GET	/api/content/public	contentController	getAllMaterials
GET	/api/content/public/:id	contentController	getPublicMaterialById
GET	/api/content/categories	contentController	getCategories
GET	/api/content/	contentController	getAllMaterials

Продолжение таблицы 3.5

Метод	Маршрут	Контроллер	Метод контроллера
GET	/api/content/generation/status	contentController	getGenerationStatus
GET	/api/content/progress	contentController	getStudentProgress
GET	/api/content/:id	contentController	getMaterialById
POST	/api/content/regeneration-quiz	contentController	regenerateQuiz
POST	/api/content/generate	contentController	uploadAndGenerate
PUT	/api/content/:id	contentController	updateMaterial
PUT	/api/content/:id/progress	contentController	updateStudentProgress
PUT	/api/content/:id/publish	contentController	publishMaterial
PUT	/api/content/:id/unpublish	contentController	unpublishMaterial
DELETE	/api/content/:id	contentController	deleteMaterial
GET	/api/admin/users	adminController	getAllUsers
PUT	/api/admin/users/role	adminController	updateUserRole
PUT	/api/admin/users/block	adminController	toggleBlock
DELETE	/api/admin/users/:id	adminController	deleteUser
GET	/api/admin/configs	adminController	getAIConfigs
PUT	/api/admin/configs	adminController	updateAIConfigs
GET	/api/admin/ai-models	adminController	getGroqModels
GET	/api/admin/categories	adminController	getCategories
POST	/api/admin/categories	adminController	addCategory
PUT	/api/admin/categories/:id	adminController	updateCategory
DELETE	/api/admin/categories/:id	adminController	deleteCategory

Представленная схема API с чётким разделением маршрутов, контроллеров и промежуточного программного обеспечения задаёт понятную структуру серверной части приложения AI-Education: каждый уровень отвечает за свою зону ответственности, что снижает взаимное «переплетение» компонентов и облегчает поиск места для доработок при развитии функционала. Такой подход повышает модульность кода, делает сопровождение проекта менее затратным для разработчика и позволяет наращивать возможности системы преимущественно за счёт добавления

новых маршрутов и обработчиков, не воспроизводя заново уже реализованную общую логику аутентификации и проверки прав доступа, сосредоточенную в middleware. В результате уменьшается риск ошибок при расширении API и сохраняется единообразие правил безопасности для всех защищённых конечных точек.

3.4 Реализация функций для гостя

3.4.1 Регистрация

Гостю доступна регистрация учётной записи: ввод ФИО, логина в виде email и пароля. На клиенте (Register.jsx) выполняется предварительная валидация полей по регулярным выражениям. На сервере обработчик register контроллера authController проверяет уникальность логина, хеширует пароль библиотекой bcryptjs [11] и создаёт запись в таблице Users с ролью обучающегося. Реализация приведена в листинге 3.7.

```
exports.register = async (req, res) => {
  const { username, password, firstName, lastName, middleName } = req.body;
  try {
    const userExists = await sql.query`SELECT 1 FROM Users
WHERE Username = ${username}`;
    if (userExists.recordset.length > 0) {
      return res.status(400).json({ message: 'Пользователь
с таким логином уже существует' });
    }

    const salt = await bcrypt.genSalt(10);
    const passwordHash = await bcrypt.hash(password, salt);

    await sql.query`INSERT INTO Users (Username,
PasswordHash, RoleId, FirstName, LastName, MiddleName,
IsBlocked)
VALUES (${username}, ${passwordHash}, 1,
${firstName}, ${lastName}, ${middleName}, 0)`;

    res.status(201).json({ message: 'Регистрация успешна'
});
  } catch (err) {
    res.status(500).json({ message: 'Ошибка сервера при
регистрации' });
  }
};
```

Листинг 3.7 – Реализация метода register

Процесс регистрации начинается с валидации входных данных на стороне сервера и проверки уникальности поля Username с помощью параметризованного SQL-запроса, чтобы исключить дублирование учётных

записей и снизить риск внедрения вредоносного текста в запрос. Пароль в открытом виде в базу не сохраняется: для записи формируются соль и хеш, что соответствует базовым требованиям к хранению учётных данных. По умолчанию новому пользователю назначается роль «Обучающийся», что отражает наиболее распространённый сценарий поступления в систему. После успешного создания записи клиентское приложение перенаправляет пользователя на страницу входа, где он может авторизоваться под созданной учётной записью.

3.4.2 Аутентификация

Для перехода от роли «Гость» к авторизованному пользователю реализован вход по паре логин (email) и пароль. По маршруту вызывается метод login контроллера authController: выполняется поиск пользователя с ролью, проверка блокировки, сравнение пароля с хешем, при успехе – выпуск JWT и возврат токена и реквизитов профиля.

Реализация метода login приведена в приложении Г.

Сначала из тела запроса извлекаются username и password. Система ищет пользователя по переданному идентификатору, после чего сравнивает введенный пароль с хешем, хранящимся в базе данных, используя bcrypt. При успешной аутентификации создается JWT токен со сроком действия 24 часа, содержащий информацию о пользователе. Клиенту возвращается токен доступа, время его истечения и базовые данные пользователя. Этот токен затем используется для аутентификации при обращении к защищенным маршрутам API.

3.4.3 Просмотр публичной информации

Гостю доступен демонстрационный просмотр учебных материалов, отмеченных преподавателем как опубликованные и публичные, без входа в систему. На главной странице приложения компонент PublicInfo запрашивает у API список таких материалов и справочник категорий. При выборе карточки загружается полное содержимое материала по идентификатору, но часть блоков на стороне интерфейса намеренно ограничена и сопровождается призывом к регистрации. Серверная реализация опирается на методы getAllMaterials, getPublicMaterialById и getCategories контроллера contentController.

Фрагменты реализации публичного доступа к каталогу приведены в приложении Д.

При отсутствии JWT в запросе метод getAllMaterials формирует выборку только записей с признаками публикации и публичности, с сортировкой по дате создания. Метод getPublicMaterialById дополнительно проверяет те же условия по идентификатору материала: если запись не найдена, клиент получает соответствующее сообщение. Метод getCategories возвращает

полный справочник категорий для построения фильтра на странице гостя. Загрузка данных на главной странице гостя представлена в листинге 3.8.

```
useEffect(() => {
  axios.get(`${API_BASE_URL}/content/public`).then(res =>
    setMaterials(res.data));
  axios.get(`${API_BASE_URL}/content/categories`).then(res =>
    setCategories(res.data));
  setIsStateHydrated(true);
}, []);

const openMaterial = (id) => {
  axios.get(`${API_BASE_URL}/content/public/${id}`).then(res
=> setSelected(res.data));
};
```

Листинг 3.8 – Загрузка данных на главной странице гостя

После получения списка материалов и категорий интерфейс выполняет клиентскую фильтрацию по строке поиска и выбранной категории. При открытии материала вызывается `openMaterial`, после чего отображаются заголовок, метаданные, фрагмент конспекта с подсказкой о полном доступе после входа, а также тренажёр терминов; блоки теста, самопроверки и экспорта для гостя не показываются, вместо них выводится пояснение и ссылки на регистрацию и вход. Таким образом третья функция гостя реализует ознакомительный просмотр публичного контента без выдачи прав, доступных только обучающемуся после аутентификации.

3.5 Реализация функций для обучающегося

3.5.1 Просмотр образовательных материалов

После авторизации обучающийся получает доступ к учебной панели, где может просматривать список доступных материалов, использовать поиск и фильтрацию, открывать нужную тему и переходить к её содержимому. На клиентской стороне эта логика реализована в компоненте `StudentDashboard`, где формируются списки `filtered` и `sorted`, а открытие материала выполняется через функцию `openMaterial`.

Пример реализации приведён в приложении Е.

Данный фрагмент показывает, что обучающийся работает с уже опубликованным учебным контентом: может найти нужную тему, отсортировать материалы и открыть выбранную лекцию для изучения.

3.5.2 Прохождение сгенерированных тестов

Внутри открытого материала обучающийся проходит тест, после чего система показывает результат и сохраняет его в прогрессе. На клиенте это реализовано в функции `checkQuiz`, где вычисляется итог по выбранным

ответам и формируется процент выполнения. Пример реализации представлен в листинге 3.9.

```
const checkQuiz = () => {

    let correct = 0;
    selected.Quizzes.forEach((q, i) => {
        const selectedIndices = Array.isArray(answers[i]) ?
[...answers[i]].sort((a, b) => a - b) : [];

        const correctIndices =
Array.isArray(q.correctAnswerIndices) ?
[...q.correctAnswerIndices].sort((a, b) => a - b) : [];
        if (selectedIndices.length === correctIndices.length &&
selectedIndices.every((v, idx) => v === correctIndices[idx])) {
            correct++;
        }
    });

    setScore(`Ваш результат: ${correct} из
${selected.Quizzes.length}`);
    setIsChecked(true);

    const percent = Math.round((correct /
selected.Quizzes.length) * 100);
    if (selected?.Id) {
        updateProgress(selected.Id, { quizCompleted: true,
quizScorePercent: percent });
    }
};
```

Листинг 3.9 – Проверка теста

Таким образом, встроенное тестирование выполняет сразу две взаимосвязанные задачи. С точки зрения обучающегося оно даёт обратную связь по усвоению темы: по результатам прохождения становится понятно, какие положения конспекта вызвали затруднения и что стоит повторить. С точки зрения системы фиксация ответов и итогового результата позволяет сохранять сведения для последующего анализа, построения статистики по материалам и отслеживания прогресса обучения во времени, что поддерживает идею непрерывного учебного процесса и повышает прозрачность взаимодействия пользователя с учебным контентом.

3.5.3 Экспорт учебных материалов

Для самостоятельной работы обучающийся может выгрузить содержимое открытого материала в текстовый файл. В экспорт включаются основные разделы: конспект, термины, блок самопроверки и практическая часть. Пример функции, которая отвечает за экспорт материалов в формат

текстового файла и включает в экспорт все разделы, `handleExport` приведён в листинге 3.10.

```
const handleExport = () => {
  const termsText = selected?.Terms?.map(t => `${t.term} -
${t.definition}`).join('\n') || 'Нет данных';
  const selfCheckText = selected?.SelfCheck?.join('\n') ||
'Нет данных';
  const practicalQuestionsText =
selected?.PracticalTask?.questions?.join('\n') || 'Нет данных';

  const text = `МАТЕРИАЛ: ${selected.Title}
КАТЕГОРИЯ: ${selected.CategoryName}
АВТОР: ${selected.LastName} ${selected.FirstName}
${selected.MiddleName}

КОНСПЕКТ:
${selected.Summary || 'Нет данных'}

ТЕРМИНЫ:
${termsText}`;

  const blob = new Blob([text], { type: 'text/plain;
charset=utf-8' });
  const link = document.createElement('a');
  link.href = URL.createObjectURL(blob);
  link.download = `${selected.Title}.txt`;
  link.click();
};
```

Листинг 3.10 – Экспорт учебных материалов

Функция экспорта позволяет обучающемуся сохранить материал в текстовом формате и использовать его вне интерфейса приложения.

3.6 Реализация функций для преподавателя

3.6.1 Загрузка исходных материалов

Подготовка учебного модуля со стороны преподавателя начинается с заполнения формы и загрузки исходного файла: в интерфейсе задаются название темы, тематическая категория, параметры содержания (в том числе объём конспекта и число вопросов теста в допустимых пределах) и выбирается документ в формате PDF или DOCX, который служит основой для последующей генерации. Такой порядок действий отражает типичный методический сценарий: сначала фиксируются учебные рамки и требования к материалу, затем подключается исходный текст. На клиентской стороне перед отправкой выполняется проверка заполненности полей и корректности выбранного файла; при успешной валидации запускается обработчик `handleUpload`, который формирует `FormData` и инициирует запрос к серверу

для создания материала. Пример реализации данной функции приведён в листинге 3.11.

```
const handleUpload = async (e) => {
  e.preventDefault();

  const normalizedTitle = title.trim();
  if (!file) return setUploadFeedback({ type: 'error', text:
'Sначала выберите PDF или DOCX файл.' });
  if (file.size === 0) return setUploadFeedback({ type:
'error', text: 'Выбранный файл пустой (0 байт).' });

  const formData = new FormData();
  formData.append('document', file);
  formData.append('title', normalizedTitle);
  formData.append('category', settings.category);
  formData.append('summaryLength', settings.summaryLength);
  formData.append('quizCount', parseInt(settings.quizCount,
10));

  await api.post('/content/generate', formData);
  await fetchList();
};
```

Листинг 3.11 – Загрузка исходного материала

Этот этап реализует переход от исходного документа к структурированному учебному материалу внутри системы.

3.6.2 Настройка параметров генерации

Перед запуском генерации преподаватель выбирает параметры, влияющие на итоговый материал: категорию, объём конспекта и количество вопросов. Это позволяет адаптировать результат под учебную задачу. Пример интерфейсной части представлен в листинге 3.12.

```
const [settings, setSettings] = useState({
  summaryLength: 'средний', quizCount: '3',
  category: ''
});<select onChange={e => setSettings({ ...settings,
summaryLength: e.target.value })}>
  <option value="средний">Средний</option>
  <option value="краткий">Краткий</option>
  <option value="подробный">Подробный</option>
</select><input
  type="number"
  value={settings.quizCount}
  onChange={e => setSettings({ ...settings,
quizCount:e.target.value })}/>
```

Листинг 3.12 – Параметры генерации

Настройка параметров генерации делает процесс подготовки материалов управляемым и гибким.

3.6.3 Редактирование сгенерированного контента

После генерации преподаватель может вручную доработать материал: изменить заголовок, основной текст, термины, вопросы самопроверки и практическую часть. Сохранение изменений выполняется через `setEditMode` и последующее обновление материала. Пример – в листинге 3.13.

```
<input value={editMode.Title} onChange={e => setEditMode({
...editMode, Title: e.target.value })} />

<textarea
  value={editMode.Summary}
  onChange={e => setEditMode({ ...editMode, Summary:
e.target.value })}
/>

<textarea
  value={editMode.SelfCheck ? editMode.SelfCheck.join('\n') :
''}
  onChange={e => setEditMode({ ...editMode, SelfCheck:
e.target.value.split('\n') })}
/>

<button className="btn-green" onClick={async () => {
  await api.put(`/content/${editMode.Id}`, editMode);
  const refreshed = await api.get(`/content/${editMode.Id}`);
  setEditMode(refreshed.data);
}}>
  Сохранить изменения
</button>
```

Листинг 3.13 – Редактирование материала

Редактирование необходимо для педагогической доработки материала перед публикацией.

3.6.4 Публикация учебных материалов

После того как сгенерированный материал прошёл проверку и при необходимости доработку, преподаватель может перевести его в опубликованное состояние: тогда запись появляется в каталоге для обучающихся и становится доступной для полноценного изучения, тестирования и учёта прогресса. Такой шаг завершает «внутренний» цикл подготовки контента и переводит материал в режим учебной эксплуатации. Если после публикации выявляются ошибки, устаревшие формулировки или требуется обновить содержание, преподавателю доступна операция снятия с

публикации, после чего материал временно скрывается из общего доступа студентов, что позволяет безопасно внести правки и при необходимости снова опубликовать обновлённую версию. Пример реализации соответствующей логики приведён в листинге 3.14.

```
{!m.IsPublished && (
  <button className="btn-green" onClick={async () => {
    await api.put(`/content/${m.Id}/publish`);
    await fetchList();
  }}>
    Опубликовать
  </button>
)}

{m.IsPublished && (
  <button className="btn-outline" onClick={async () => {
    await api.put(`/content/${m.Id}/unpublish`);
    await fetchList();
  }}>
    Снять с публикации
  </button>
)}
```

Листинг 3.14 – Публикация и снятие с публикации

Публикация завершает цикл подготовки материала и делает его частью учебного процесса.

3.7 Реализация функция для администратора

3.7.1 Управление учетными записями

Администратор контролирует пользовательские записи: просматривает список, меняет роли, блокирует и удаляет аккаунты. Эти действия сосредоточены в AdminDashboard и позволяют поддерживать порядок в системе. Пример – в листинге 3.15.

```
const handleRoleChange = async (userId, newRoleId) => {
  await api.put('/admin/users/role', { userId, newRoleId });
  fetchData();
};

const handleToggleBlock = async (userId, currentStatus) => {
  await api.put('/admin/users/block', { userId, isBlocked:
!currentStatus });
  fetchData();
};

const handleDeleteUser = async (targetUser) => {
  await api.delete(`/admin/users/${targetUser.Id}`);
  fetchData();
};
```

Листинг 3.15 – Управление пользователями

За счёт этих операций администратор обеспечивает корректное распределение прав и контроль доступа.

3.7.2 Конфигурация параметров AI-моделей

Администратор управляет параметрами генерации контента на уровне всей системы: выбирает модель, стиль формирования материала и числовые параметры (Groq [14]). Изменения сохраняются в SystemConfigs и влияют на работу генерации у преподавателей. Пример – в листинге 3.16.

```
const [localSettings, setLocalSettings] = useState({
  AI_MODEL: '', SYSTEM_PROMPT: '', AI_TEMPERATURE: '',
  MAX_TOKENS: ''
});

const handleSaveAISettings = async () => {
  const promises = Object.entries(localSettings).map(([key,
value]) =>
    api.put('/admin/configs', { key, value: value.toString()
  })
);
  await Promise.all(promises);
  fetchData();
};
```

Листинг 3.16 – Настройки ИИ

Данный функционал позволяет администратору централизованно настраивать качество и стиль генерируемых учебных материалов.

3.8 Реализация промежуточных обработчиков

Промежуточные обработчики (middleware) в серверной части приложения AI-Education выполняют общие действия до передачи управления контроллерам: проверка учётной записи по токenu, ограничение доступа по роли, приём файла учебного документа при генерации материала. Такой подход уменьшает дублирование кода в контроллерах и делает правила доступа наглядными.

3.8.1 Аутентификация и проверка роли

Для защищённых разделов API используется модуль authMiddleware, в котором реализованы функции verifyToken и requireRole. Первая извлекает токен из заголовка запроса, проверяет его подпись и срок действия, дополнительно сверяет состояние пользователя в базе данных (наличие записи и признак блокировки), после чего помещает расшифрованные сведения о пользователе в объект req.user. Вторая сравнивает роль из req.user с

допустимым списком ролей для конкретного маршрута и при несовпадении прекращает дальнейшую обработку. Реализация приведена в листинге 3.17.

```
const jwt = require('jsonwebtoken');
const { sql } = require('../config/db');
const verifyToken = async (req, res, next) => {
  const authHeader = req.header('Authorization');
  const token = authHeader?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Токен отсутствует.' });
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const result = await sql.query`SELECT IsBlocked FROM Users WHERE Id = ${decoded.userId}`;
    const dbUser = result.recordset[0];
    if (!dbUser) return res.status(401).json({ message: 'Пользователь не найден.' });
    if (dbUser.IsBlocked) return res.status(403).json({ message: 'Аккаунт заблокирован.' });
    req.user = decoded;
    next();
  } catch (error) {
    console.error('Ошибка верификации токена:', error.message);
    res.status(401).json({ message: 'Сессия истекла. Войдите заново.' });
  }
};
const requireRole = (rolesArray) => (req, res, next) => {
  if (!rolesArray.includes(req.user.role)) {
    return res.status(403).json({ message: 'Недостаточно прав.' });
  }
  next();
};
module.exports = { verifyToken, requireRole };
```

Листинг 3.17 – Промежуточные обработчики verifyToken и requireRole

Обработчик verifyToken ожидает, что клиент передал токен в стандартном виде «тип токена и сама строка токена» в заголовке запроса. Если строка отсутствует, повреждена или срок её действия истёк, пользователь получает сообщение о необходимости повторного входа. После успешной проверки подписи выполняется обращение к таблице Users: таким образом даже при корректном токене доступ блокируется, если учётная запись удалена или помечена как заблокированная. Данные из токена (идентификатор, роль, ФИО и логин) попадают в req.user, и далее контроллеры могут опираться на них без повторного разбора токена.

Обработчик `requireRole` принимает массив допустимых имён ролей (например, преподаватель и администратор для генерации материала, только обучающийся для сохранения прогресса). Он выполняется после `verifyToken`, когда объект `req.user` уже заполнен. Если роль пользователя не входит в разрешённый перечень, возвращается ответ о недостаточных правах. В приложении это разделяет возможности гостя, обучающегося, преподавателя и администратора без дублирования проверок внутри каждого контроллера.

3.8.2 Загрузка исходных файлов

Загрузка файла PDF или DOCX для последующей генерации учебного материала выполняется с помощью библиотеки `multer` [12]. Настройка задаёт каталог сохранения на диске, правило имени файла и фильтр по типу содержимого, чтобы на сервер попадали только разрешённые форматы. Сконфигурированный объект `upload` подключается в цепочку обработчиков вместе с `verifyToken` и `requireRole`, так что загрузка доступна только авторизованным пользователям с подходящей ролью. Реализация приведена в листинге 3.18.

```
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + '-' + file.originalname);
  }
});

const fileFilter = (req, file, cb) => {
  const allowedTypes = [
    'application/pdf',
    'application/vnd.openxmlformats-officedocument.wordprocessingml.document'
  ];

  if (allowedTypes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error('Разрешены только файлы форматов PDF и DOCX!'), false);
  }
};

const upload = multer({ storage: storage, fileFilter: fileFilter });
module.exports = upload;
```

Листинг 3.18 – Настройка приёма файла документа

В объекте `storage` задано, что файлы сохраняются в каталог `uploads`, а имя формируется из текущего времени и исходного имени файла, чтобы снизить риск совпадения имён при многократных загрузках. Функция `fileFilter` проверяет MIME-тип входящего файла: допускаются только PDF и DOCX. Итоговый объект `upload` используется в маршруте генерации материала в режиме приёма одного файла с полем `document`, после чего контроллер `contentController` получает путь к сохранённому файлу и передаёт его в цепочку разбора документа и обращения к ИИ.

3.9 Структура клиентской части

Клиентская часть реализована на React с компонентным подходом: интерфейс собран из страниц-экранов и вспомогательных модулей. Основная логика и ресурсы сосредоточены в каталоге `frontend/src`. Там размещены страницы приложения, общий HTTP-клиент, контекст аутентификации, переиспользуемые UI-элементы и глобальные стили. Структура отражает разделение по ролям (гость, студент, преподаватель, администратор) и по функциям: публичный просмотр материалов, учебная работа со сгенерированным контентом, подготовка материалов с помощью ИИ, администрирование системы и учётной записи.

Содержимое папки `src` представлено в таблице 3.6.

Таблица 3.6 – Содержимое папки `src`

Директория	Назначение
<code>pages</code>	Страницы приложения: вход, регистрация, публичная витрина, панели студента, преподавателя и администратора, профиль пользователя.
<code>api</code>	Настройка HTTP-клиента (<code>axios</code>): базовый URL, перехватчики запросов (например, JWT), единая точка обращения к REST API сервера.
<code>context</code>	React Context для хранения состояния текущего пользователя, токена, методов входа/выхода.
<code>components</code>	Переиспользуемые UI-компоненты, не привязанные к одной странице (например, модальное подтверждение действия).
<code>utils</code>	Вспомогательные функции для клиента (уведомления пользователю).

Продолжение таблицы 3.6

Директория	Назначение
App.jsx	Корневой компонент: макет шапки, навигация, объявление маршрутов react-router-dom.
main.jsx	Точка входа: монтирование приложения в DOM, обёртка в AuthProvider и StrictMode.
index.css	Глобальные стили, оформление карточек материалов, форм, кнопок.

В корне src файлы App.jsx, main.jsx и index.css задают композицию приложения, подключение провайдера авторизации и визуальное оформление. Main.jsx подключает App.jsx как корень дерева компонентов. Компоненты и их описания представлены в таблице 3.7.

Таблица 3.7 – Компоненты и их описания

Компонент	Роль	Назначение компонента
PublicInfo	Гость, авторизованный пользователь (при переходе на главную страницу)	Публичная информация о системе и каталог опубликованных учебных материалов с кратким просмотром конспекта.
Login	Гость	Форма входа по логину и паролю, редирект на /dashboard после успешной аутентификации.
Register	Гость	Форма регистрации нового пользователя.
TeacherDashboard	Преподаватель	Загрузка PDF/DOCX, параметры генерации (объём конспекта, число вопросов теста), вызов ИИ, просмотр и редактирование сгенерированного конспекта, терминов, тестов и ситуационной задачи, публикация материала.
StudentDashboard	Студент	Каталог доступных материалов, просмотр конспекта, изучение терминов, прохождение теста, самопроверка, ситуационная задача, отметка прогресса по материалу, экспорт материала.

Продолжение таблицы 3.7

Компонент	Роль	Назначение компонента
AdminDashboard	Администратор	Вкладки: пользователи (роли, блокировка), категории знаний, настройки ИИ (модель, промпт, температура, лимит токенов), просмотр материалов, управление публикацией.
Profile	Студент, преподаватель, администратор	Личный кабинет: данные профиля, смена пароля и логина
ConfirmDialog	Преподаватель, администратор	Универсальное модальное окно подтверждения действий

Маршрутизация реализована средствами react-router-dom: в App.jsx объявлены соответствующие маршруты. Неавторизованный пользователь при попытке открыть защищённые разделы перенаправляется на страницу входа. Административные функции доступны только роли «Администратор». Роли «Преподаватель» и «Обучающийся» разделены отдельными панелями.

В совокупности страницы и модули src обеспечивают полный цикл для гостя (ознакомление и вход в систему), студента (работа с опубликованными материалами и фиксация прогресса), преподавателя (генерация и редактирование учебных модулей на основе ИИ) и администратора (пользователи, категории, конфигурация нейросети и контроль материалов).

3.9 Выводы по разделу

1. Доступ к данным реализован через официальный драйвер mssql и параметризованные SQL-запросы. Такой подход даёт прямой контроль над схемой и запросами к MS SQL Server, что удобно для учебного проекта с явно заданной структурой таблиц и хранимой логикой на стороне БД.

2. Серверная часть построена на Express с разделением по слоям: маршруты (routes) подключают контроллеры с бизнес-логикой, обращение к БД инкапсулировано в запросах из контроллеров, вынесены middleware (аутентификация по JWT, загрузка файлов). Это обеспечивает модульность API (авторизация, контент, администрирование) и упрощает сопровождение кода.

3. Реализована аутентификация и авторизация: пароли хранятся в виде хеша, для сессий используются JSON Web Tokens (JWT). Промежуточные обработчики проверяют токен и роль пользователя, ограничивая доступ к маршрутам генерации материалов, правкам и админским операциям.

4. Для администратора реализован набор функций управления системой: учётные записи и роли, категории знаний, конфигурация ИИ (модель, системный промпт, температура, лимит токенов), просмотр и

модерация учебных материалов. Для преподавателя – полный цикл работы с материалом: загрузка PDF/DOCX (multer), вызов сервиса генерации конспекта, тестов и заданий, редактирование и публикация.

5. Клиентское приложение реализовано на React (сборка Vite) с компонентным подходом, интерфейс оформлен собственными стилями и вспомогательными библиотеками (react-router-dom для маршрутизации, axios [13] для HTTP, уведомления пользователю). Навигация по разделам и выбор панели (студент, преподаватель, администратор) привязаны к маршрутам и роли пользователя, что явно разделяет сценарии использования системы.

4 Тестирование web-приложения

Для проверки работы всех функций пользователей web-приложения было проведено ручное тестирование. Номера функций взяты из диаграммы вариантов использования, представленной в приложении А. Описание и итоги тестирования представлены в таблице 4.1.

Таблица 4.1 – Тестирование web-приложения

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
Регистрация (1)	Гость	В правом верхнем углу нажать на кнопку «Регистрация». В поле фамилия ввести значение «Иванов», в поле имя ввести значение «Иван», в поле отчество ввести значение «Иванович», в поле логина ввести значение «test@gmail.com», в поле пароль ввести значение «Ivan123!», нажать на кнопку «Создать аккаунт».	Автоматический переход на форму авторизации, успешная регистрация пользователя.	Пройден
Аутентификация (2)	Гость	В правом верхнем углу нажать на кнопку «Вход». В поле логина ввести значение «test@gmail.com», в поле пароля ввести значение «Ivan123!», нажать на кнопку «Войти».	Автоматический переход в раздел «Учебная панель», появление в правом верхнем углу кнопки «Выйти» и раздела «Профиль».	Пройден
Просмотр публичной информации (3)	Гость	Перейти на вкладку «Главная» и выбрать один из доступных для просмотра материал и нажать	Открывается карточка материала: виден фрагмент конспекта с	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
		на кнопку «Открыть».	затемнением и надпись «Продолжение конспекта доступно после входа в систему», отображается блок «Тренажер терминов» с карточками, внизу блок «Заинтересовал материал?» со ссылками «Зарегистрироваться» и «Войти».	
Конфигурация параметров AI-моделей (4)	Администратор	В правом верхнем углу нажать на кнопку «Войти», в поле логина ввести значение «administrator@gmail.com», в поле пароля ввести значение «Admin123. Перейти на «Панели администратора» на вкладку «Настройки ИИ». Изменить, например, поле «Температура» (0-1), нажать «Сохранить настройки ИИ».	Появляется уведомление об успешном сохранении. При повторном открытии вкладки значения соответствуют сохраненным.	Пройден
Управление материалами (5)	Администратор	В панели администратора открыть вкладку «Материалы». Убедиться, что загружается таблица/список всех	Отображается перечень учебных материалов с возможностью дальнейших действий (поиск, фильтр,	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
		материалов системы (тема, категория, автор, даты добавления и изменения).	сортировка, просмотр, удаление, снятие с публикации).	
Управление материалами (5)	Администратор	В панели администратора открыть вкладку «Материалы». Убедиться, что загружается таблица/список всех материалов системы (тема, категория, автор, даты добавления и изменения).	Отображается перечень учебных материалов с возможностью дальнейших действий (поиск, фильтр, сортировка, просмотр, удаление, снятие с публикации).	Пройден
Поиск материала (6)	Администратор	На вкладке «Материалы» ввести в поле поиска часть названия темы или ФИО автора, дождаться фильтрации списка.	В списке остаются только материалы, удовлетворяющие строке поиска.	Пройден
Удаление материала (7)	Администратор	На вкладке «Материалы» у выбранного материала нажать «Удалить», в диалоге подтверждения подтвердить удаление.	Материал исчезает из списка, отображается уведомлении об успешном удалении.	Пройден
Фильтрация материалов (8)	Администратор	На вкладке «Материалы» в выпадающем списке категории выбрать конкретную категорию (не «Все категории»).	Список материалов сокращается до выбранной категории.	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
Сортировка материалов (9)	Администратор	На вкладке «Материалы» изменить значение поля сортировки (например, «По названию (А-Я)» / «Сначала новые»).	Порядок строк в списке меняется в соответствии с выбранным критерием.	Пройден
Снятие материала с публикации (10)	Администратор	На вкладке «Материалы» у опубликованного материала нажать «Снять с публикации», подтвердить действие.	Статус материала меняется на неопубликованный. Студенты перестают видеть его в общем каталоге.	Пройден
Просмотр материала (11)	Администратор	На вкладке «Материалы» нажать «Просмотр» у выбранного материала.	Открывается полный просмотр конспекта, терминов, тестов и прочих полей материалов. Доступна кнопка «Назад».	Пройден
Управление учётными записями пользователей (12)	Администратор	Открыть вкладку «Пользователи».	Отображается таблица пользователей с колонками ФИО, логин, роль и элементами управления.	Пройден
Поиск пользователя (13)	Администратор	На вкладке «Пользователи» ввести в поле поиска фрагмент ФИО или логина.	Список пользователей фильтруется по введенной строке.	Пройден
Сортировка пользователей (14)	Администратор	На вкладке «Пользователи» изменить сортировку (например, «ФИО (Я-А)»).	Порядок строк в таблице меняется согласно выбранному варианту.	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
Блокировка пользователя (15)	Администратор	На вкладке «Пользователи» у пользователя с любой ролью нажать действие блокировки, подтвердить.	Пользователь помечается как заблокированный. При попытке входа под ним отображается сообщение о блокировке.	Пройден
Фильтрация пользователей (16)	Администратор	На вкладке «Пользователи» в фильтре ролей выбрать, например, только «Студенты».	В таблице остаются пользователи выбранной роли.	Пройден
Удаление пользователя (17)	Администратор	На вкладке «Пользователи» у пользователя без материалов и не являющегося последним админом нажать «Удалить», подтвердить.	Запись пользователя удаляется из списка, показывается уведомление об успехе.	Пройден
Смена роли пользователя (18)	Администратор	На вкладке «Пользователи» в строке пользователя изменить значение в выпадающем списке роли.	Роль пользователя в таблице обновляется, показывается уведомление о смене роли. При следующем входе пользователь попадает в соответствующую панель.	Пройден
Разблокировка пользователя (19)	Администратор	На вкладке «Пользователи» у ранее заблокированного пользователя выполнить разблокировку (кнопка/действие в интерфейсе).	Статус блокировки снимается, вход под учётной записью снова возможен.	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
Управление категориями (20)	Администратор	Открыть вкладку «Категории».	Отображается список категорий и элементы добавления/редактирования/удаления.	Пройден
Фильтрация категорий (21)	Администратор	На вкладке «Категории» выбрать фильтр использования (например, только используемые / неиспользуемые).	Список категорий сокращается в соответствии с фильтром.	Пройден
Сортировка категорий (22)	Администратор	На вкладке «Категории» изменить поле сортировки категорий (например, по названию).	Порядок строк в списке категорий меняется.	Пройден
Поиск категории (23)	Администратор	На вкладке «Категории» ввести в строку поиска часть названия.	Отображаются только категории, подходящие под поиск.	Пройден
Редактирование категории (24)	Администратор	На вкладке «Категории» нажать кнопку «Редактировать» у категории, изменить название, сохранить.	Название категории обновляется в списке и в связанных материалах (в выпадающих списках).	Пройден
Удаление категории (25)	Администратор	На вкладке «Категории» удалить категорию, не привязанную к материалам (или после переноса материалов), подтвердить.	Категория удалена, при попытке удалить занятую категорию – сообщение об ошибке.	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
Добавление категории (26)	Администратор	На вкладке «Категории» в поле новой категории ввести допустимое название (буквы, дефис, пробел), нажать «Добавить».	Категория появляется в списке, отображается уведомление об успехе.	Пройден
Управление профилем (27)	Обучающийся, Администратор	В правом верхнем углу нажать на кнопку «Войти», в поле логина ввести значение «administrator@gmail.com» или «bezbashen@gmail.com», в поле пароля ввести значение «Admin123!» или «VanVan123!». Войти как администратор или студент. Перейти по ссылке «Профиль» в шапке.	Открывается страница профиля с отображением ФИО, логина, роли.	Пройден
Просмотр информации о пользователе (28)	Обучающийся, Администратор	На странице «Профиль» просмотреть блок с данными пользователя (без изменений).	Отображаются актуальные ФИО, логин и роль.	Пройден
Изменение логина (29)	Обучающийся, Администратор	На «Профиле» в форме смены логина ввести новый корректный email, нажать кнопку сохранения логина.	Сообщение об успехе. При необходимости токен обновляется, в шапке отображается новый логин после обновления данных.	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
Изменение пароля (30)	Обучающийся, Администратор	На «Профиле» заполнить поля текущего и нового пароля (с соблюдением правил сложности), подтверждение, отправить форму.	Сообщение об успешной смене пароля, вход со старым паролем невозможен.	Пройден
Настройка параметров генерации (31)	Преподаватель	В правом верхнем углу нажать на кнопку «Войти», в поле логина ввести значение «gaubars@gmail.com», в поле пароля ввести значение «Rauba123!». Войти как преподаватель, на «Учебной панели» в блоке «Загрузка новых материалов» заполнить название темы, выбрать категорию, объём конспекта, число вопросов в допустимых пределах, выбрать файл.	Поля принимают значения, при нарушении ограничений (длина названия, число вопросов) отображаются сообщения об ошибке.	Пройден
Загрузка исходных материалов (32)	Преподаватель	На «Учебной панели» в блоке «Загрузка новых материалов» нажать «Выберите файл», указать допустимый PDF или DOCX, убедиться, что имя файла отображается в форме.	Файл выбран и будет отправлен вместе с формой генерации.	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
Генерация учебного материала (33)	Преподаватель	На «Учебной панели» в блоке «Загрузка новых материалов» при корректно заполненной форме и выбранном файле нажать «Сгенерировать учебный материал».	После успешной генерации материал появляется в списке, при ошибке ИИ – сообщение с текстом ошибки.	Пройден
Удаление учебного материала (34)	Преподаватель	На «Учебной панели» в списке своих материалов нажать «Удалить», в диалоге подтвердить.	Материал удалён из списка, показано уведомление.	Пройден
Снятие с публикации (35)	Преподаватель	На «Учебной панели» у опубликованного своего материала нажать действие снятия с публикации, подтвердить.	Материал становится неопубликованным для студентов.	Пройден
Публикация учебного материала (36)	Преподаватель	На «Учебной панели» у неопубликованного своего материала (помечен как «Черновик») нажать «Опубликовать».	Материал отображается студентам в каталоге, уведомление об успехе.	Пройден
Редактирование учебного материала (37)	Преподаватель	На «Учебной панели» в списке материалов нажать кнопку «Правка» у своего материала, изменить, например, фрагмент конспекта или формулировку вопроса теста, нажать «Сохранить изменения».	Изменения сохраняются в БД, в списке отображается признак «Изменено» (если даты различаются).	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
Доступ для гостей (38)	Преподаватель	На «Учебной панели» в списке материалов нажать кнопку «Правка» у своего материала, в режиме правки установить флажок публичного доступа к материалу для гостей, сохранить.	Значение IsPublic сохраняется, на главной гость видит/не видит расширенный превью в соответствии с правилами.	Пройден
Регенерация вопроса теста (39)	Преподаватель	На «Учебной панели» в списке материалов нажать кнопку «Правка» у своего материала, в режиме редактирования материала в блоке тестов нажать регенерацию для конкретного вопроса, дождаться ответа API.	Вопрос заменяется новым, уведомление об успехе или ошибке.	Пройден
Управление профилем (40)	Преподаватель	Войти как преподаватель, перейти в «Профиль».	Открывается та же страница профиля, что и у студента с администратором, с данными преподавателя.	Пройден
Просмотр информации о пользователе (41)	Преподаватель	На «Профиле» просмотреть блок данных без изменений.	Отображаются корректные ФИО, логин, роль «Учитель».	Пройден
Изменение логина (42)	Преподаватель	На «Профиле» в форме смены логина ввести новый корректный email, нажать кнопку сохранения логина.	Логин обновлён, успешное уведомление.	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
Изменение пароля (43)	Преподаватель	На «Профиле» заполнить поля текущего и нового пароля, подтверждение, отправить форму.	Пароль обновлён, успешное уведомление	Пройден
Просмотр образовательных материалов (44)	Преподаватель	На «Учебной панели» просмотреть список материалов (вкладки «Мои материалы» / «Другие преподаватели»).	Отображаются карточки материалов с метаданными.	Пройден
Фильтрация материалов (45)	Преподаватель	На «Учебной панели» в списке материалов выбрать категорию в фильтре.	Отображаются только материалы выбранной категории.	Пройден
Поиск материала (46)	Преподаватель	На «Учебной панели» ввести текст в поле поиска по названию или ФИО автора.	Выводятся материалы, которые соответствуют поисковой строке.	Пройден
Сортировка материалов (47)	Преподаватель	На «Учебной панели» изменить параметр сортировки списка (например, по дате).	Порядок карточек меняется (дата, название).	Пройден
Просмотр материала (48)	Преподаватель	На «Учебной панели» нажать «Просмотр» у материала (свой или чужой опубликованный – по правилам доступа).	Открывается режим просмотра: полный конспект, термины, тесты, самопроверка, ситуационная задача.	Пройден
Прохождение сгенерированных тестов (49)	Преподаватель	На «Учебной панели» на вкладке материалов, которые были	Отображается итоговая строка с результатом и \	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
		опубликованы другими пользователями, в просмотре материала отметить варианты ответов, нажать «Проверить тест».	кнопка «Пройти заново».	
Экспорт материала в текстовый файл (50)	Преподаватель	На «Учебной панели» в просмотре материала нажать «Экспорт».	Загружается .txt с конспектом, терминами, вопросами для самопроверки и ситуационной задачей.	Пройден
Просмотр образовательных материалов (51)	Обучающийся	На «Учебной панели» студента прокрутить список опубликованных материалов.	Отображаются карточки с категорией, названием, автором, датами.	Пройден
Фильтрация материалов (52)	Обучающийся	На «Учебной панели» студента выбрать категорию в выпадающем списке над сеткой материалов.	Остаются материалы только выбранной категории.	Пройден
Сортировка материалов (53)	Обучающийся	На «Учебной панели» студента изменить поле сортировки («Сначала новые»).	Порядок карточек соответствует выбранному критерию.	Пройден
Поиск материала (54)	Обучающийся	На «Учебной панели» студента ввести подстроку в поле поиска по названию или ФИО автора.	Список фильтруется по введённому тексту.	Пройден
Переход к последнему материалу (55)	Обучающийся	На «Учебной панели» студента в блоке «Продолжить обучение» при	Открывается тот же материал, что и в последней сессии	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
		наличии последнего материала нажать «Продолжить».	(по данным прогресса).	
Просмотр мини-статистики (56)	Обучающийся	На «Учебной панели» студента на главной части панели студента просмотреть блок «Мини-статистика».	Отображен следующий блок: открыто материалов, попыток тестов, средний %.	Пройден
Просмотр материала (57)	Обучающийся	На «Учебной панели» студента нажать «Открыть» у любого доступного материала.	Открывается полный конспект, блок прогресса, термины, тест, самопроверка, ситуационная задача и кнопка «Экспорт».	Пройден
Учёт прогресса по материалу (58)	Обучающийся	На «Учебной панели» студента нажать «Открыть» у любого доступного материала, внутри материала отметить чекбоксы «Конспект прочитан», «Термины изучены», «Тест пройден» по очереди.	Счётчик «Выполнено шагов» обновляется, после обновления страницы состояние сохраняется.	Пройден
Прохождение сгенерированных тестов (59)	Обучающийся	На «Учебной панели» студента нажать «Открыть» у любого доступного материала выбрать варианты ответов теста по всем вопросам, нажать «Проверить».	Показывается строка результата и кнопка «Заново», обновляется статистика попыток.	Пройден
Экспорт материала в текстовый файл (60)	Обучающийся	На «Учебной панели» студента нажать «Открыть» у любого доступного	Скачивается текстовый файл с содержимым модуля.	Пройден

Продолжение таблицы 4.1

Функция	Роль	Ход тестирования	Ожидаемый результат	Итог
		материала и в режиме просмотра материала нажать «Экспорт».		

4.1 Выводы по разделу

1. Проведено ручное тестирование всех ключевых функций web-приложения. Корректность работы системы подтверждена соответствием фактических результатов тестирования ожидаемым. Количество тестов составило 60.

5 Руководство пользователя

5.1 Руководство гостя

При открытии сайта гость автоматически попадает на главную страницу, которая служит визитной карточкой системы генерации образовательного контента. На этой странице представлена публичная информация в виде материалов, которые доступны в режиме ограниченного просмотра.

Главная страница представлена на рисунке 5.1.

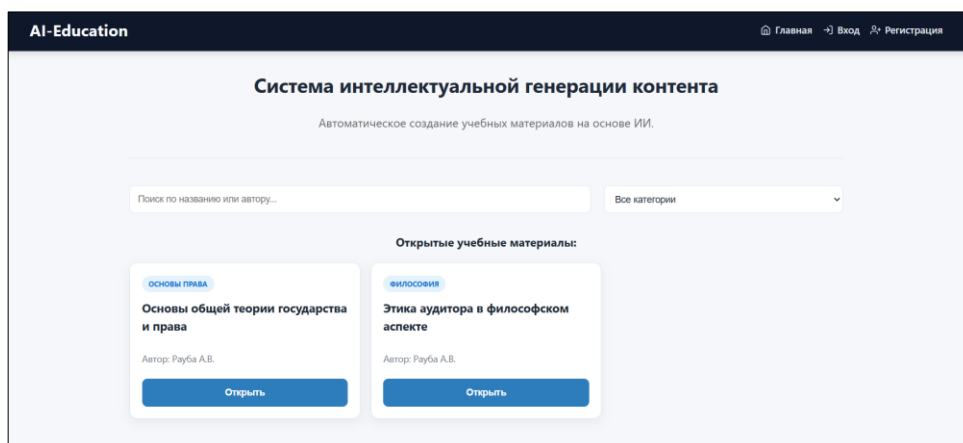


Рисунок 5.1 – Главная страница

На главной странице доступна также фильтрация и поиск открытых на данный момент для гостя материалов.

5.1.1 Регистрация

Если гость заинтересован в том, чтобы взаимодействовать с материалами, которые были сгенерированы ИИ, он может нажать на кнопку «Регистрация», которая перенаправит его на страницу регистрации.

Форма регистрации представлена на рисунке 5.2.

Рисунок 5.2 – Форма регистрации

Здесь необходимо заполнить форму, указав свои ФИО, логин и пароль. После заполнения формы, гость нажимает кнопку «Создать аккаунт», которая инициирует процесс проверки введенных данных.

После успешного ввода данных, и нажатия на кнопку «Создать аккаунт», будет создан личный аккаунт в системе. По умолчанию у пользователя будет роль «Студент».

После пользователь получает доступ к полному функционалу web-приложения, включая возможность просмотра всех опубликованных материалов, прохождения сгенерированных тестов и экспорта материалов.

5.1.2 Просмотр публичной информации

На главной странице при нажатии на кнопку «Открыть» какого-либо материала, гостю будет видна часть сгенерированного конспекта, блок. Пример открытого материала на главной странице представлен на рисунке 5.3.

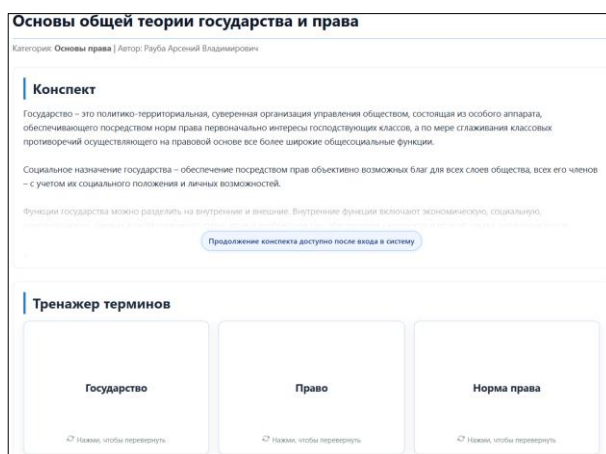


Рисунок 5.3 – Открытый материал на главной странице

Блок, который призывает зарегистрироваться или авторизоваться, чтобы в полной мере пользоваться материалами системы интеллектуальной генерации, представлен на рисунке 5.4.

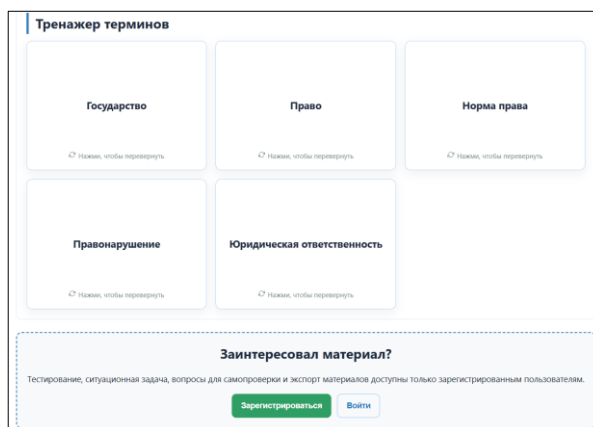


Рисунок 5.4 – Блок с информационным сообщением

5.1.3 Аутентификация

После того, как гость зарегистрировался, ему доступна возможность авторизации. Форма авторизации представлена на рисунке 5.5.

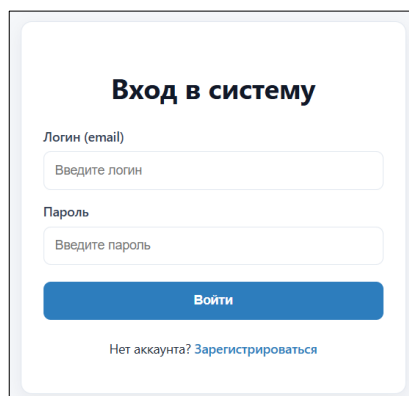
The image shows a login form with a light blue border. At the top, the title 'Вход в систему' is centered in bold. Below it, there are two input fields: the first is labeled 'Логин (email)' with a placeholder 'Введите логин', and the second is labeled 'Пароль' with a placeholder 'Введите пароль'. Below these fields is a blue button with the text 'Войти'. At the bottom, there is a link that says 'Нет аккаунта? Зарегистрироваться'.

Рисунок 5.5 – Форма авторизации

После того, как пользователь заполнит данную форму своими данными, ему станет доступна учебная панель с опубликованными материалами и мини-статистикой.

5.2 Руководство обучающегося

5.2.1 Аутентификация

Если у обучающегося уже есть зарегистрированный аккаунт, ему необходимо перейти на страницу авторизации.

На этой странице обучающийся может ввести свой логин и пароль в соответствующие поля формы авторизации, чтобы получить доступ к личному кабинету и всем функциям системы интеллектуальной генерации и управления образовательным контентом.

Форма авторизации представлена на рисунке 5.6.

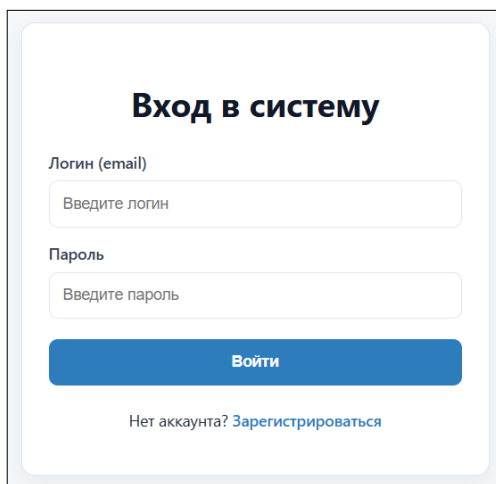
The image shows a login form with a light blue border. At the top, the title 'Вход в систему' is centered in bold. Below it, there are two input fields: the first is labeled 'Логин (email)' with a placeholder 'Введите логин', and the second is labeled 'Пароль' with a placeholder 'Введите пароль'. Below these fields is a blue button with the text 'Войти'. At the bottom, there is a link that says 'Нет аккаунта? Зарегистрироваться'.

Рисунок 5.6 – Форма авторизации

Учебная панель студента после входа представлена на рисунке 5.7.

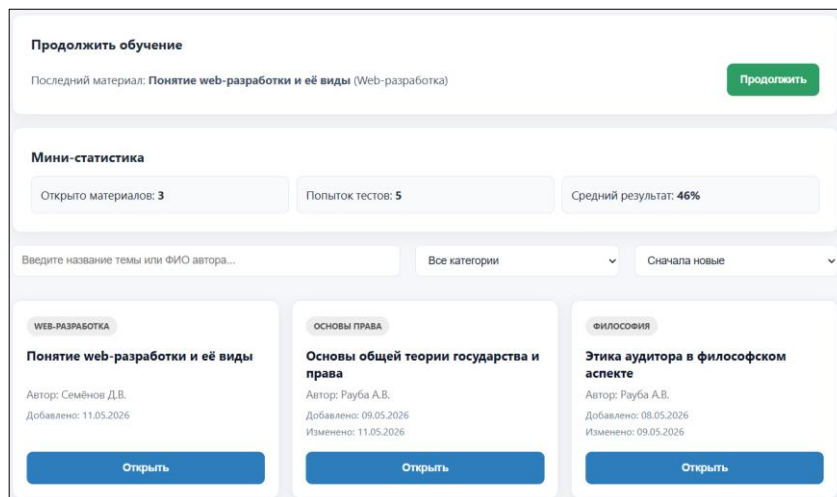


Рисунок 5.7 – Учебная панель обучающегося

Теперь обучающийся сможет получить доступ ко всем опубликованным материалам в полном объеме, тестам, терминам, вопросам для самопроверки, ситуационной задаче.

5.2.2 Прохождение сгенерированных тестов

Студенту доступны тесты, которые, как и конспект, являются сгенерированными ИИ и все вопросы генерируются на основе получившегося текста. Пример такого теста и его прохождение представлено на рисунке 5.8.

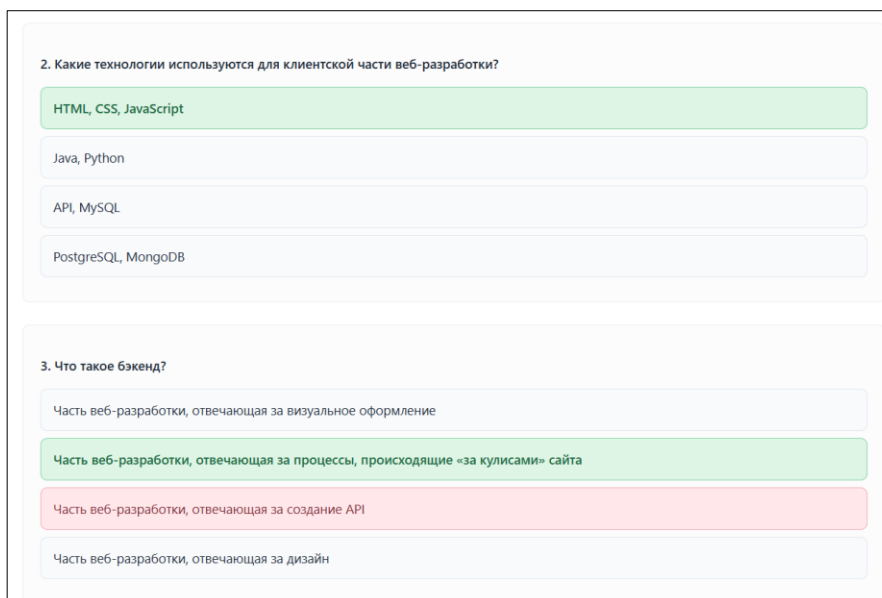


Рисунок 5.8 – Прохождение сгенерированного теста

После того, как данный тест будет пройден, можно увидеть результат и пройти его заново. Также будет автоматически поставлена отметка в прогрессе

по материалу о том, что тест пройден и данный результат пойдет в личную мини-статистику.

5.2.3 Экспорт учебных материалов

Студенту также доступна возможность экспорта учебных материалов в текстовый файл. Содержимое текстового файла представлено на рисунке 5.9.

МАТЕРИАЛ: Понятие веб-разработки и ее виды
КАТЕГОРИЯ: веб-разработка
АВТОР: Семенов Даниил Вячеславович

КОНСПЕКТ:
Веб-разработка – это процесс создания сайтов и веб-приложений, функционирующих в интернете. Она включает в себя фронтенд и бэкэнд разработку, что позволяет создавать интерактивные и функциональные решения. Веб-разработка применяется во многих областях и является востребованной профессией.

Веб-разработка включает в себя множество технологий и языков программирования, таких как HTML, CSS, JavaScript, Java и Python. Эти инструменты позволяют веб-разработчикам создавать структуры сайтов, оформлять их визуально и добавлять интерактивные элементы.

Разработка сайтов требует участия команды специалистов, что позволяет значительно ускорить процесс и повысить его эффективность. Процесс разработки разбивается на несколько этапов, что способствует четкому распределению задач и улучшению координации работы команды.

Веб-разработка становится все более популярной, и все больше людей выбирают эту профессию. Причины этого тренда очевидны: с ростом цифровых технологий увеличивается потребность в качественных веб-сайтах и веб-приложениях.

Веб-разработка предлагает множество возможностей для профессионального роста и развития. Специалисты в этой области могут работать как в крупных компаниях, так и как фрилансеры, что дает им гибкость в выборе рабочего графика и проектов.

Профессия веб-разработчика востребована на рынке труда, и с увеличением числа стартапов и онлайн-бизнесов растет спрос на специалистов, способных создавать и поддерживать веб-решения.

Веб-разработка делится на фронтенд и бэкэнд, а также включает в себя разработку пользовательских интерфейсов, серверной логики и баз данных. Бэкэнд-разработка играет ключевую роль в функционировании веб-ресурса, управляет обработкой запросов, управлением данными и обеспечением безопасности информации.

ТЕРМИНЫ:
Веб-разработка – процесс создания сайтов и веб-приложений, функционирующих в интернете.
Фронтенд – часть веб-разработки, отвечающая за визуальное оформление и взаимодействие с пользователем.
Бэкэнд – часть веб-разработки, отвечающая за процесс, происходящий «за кулисами» сайта, включая обработку запросов, управление данными и обеспечение безопасности информации.
HTML – язык разметки, используемый для создания структуры сайтов.
CSS – язык стилей, используемый для оформления сайтов визуально.
JavaScript – язык программирования, используемый для добавления интерактивных элементов на сайты.
API – интерфейс программирования приложений, обеспечивающий взаимодействие между клиентской частью и сервером.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ:
Что такое фронтенд и бэкэнд в веб-разработке?
Какие технологии используются для клиентской части веб-разработки?
Какие задачи стоят перед бэкэнд-разработчиками?
Что такое API и для чего он используется?
Почему веб-разработка является востребованной профессией?

Рисунок 5.9 – Содержимое текстового файла

Данная опция очень удобна, ведь нужный материал можно хранить на физическом устройстве и обращаться к нему в любое время, не переходя на сайт.

5.3 Руководство преподавателя

5.3.1 Аутентификация

Если преподаватель не зашёл в аккаунт, то ему необходимо перейти на страницу авторизации, где он сможет войти, используя свой логин и пароль. Данная процедура является абсолютно идентичной для всех ролей. Однако стоит сказать, что роль учителя назначается пользователю через панель администратора. Учебная панель преподавателя представлена на рисунке 5.10.

Рисунок 5.10 – Учебная панель преподавателя

Как видно на рисунке, на данной панели происходит полный цикл создания материала, от его генерации до публикации.

5.3.2 Настройка параметров генерации и загрузка исходных материалов

На рисунке 5.11 представлена настройка параметров генерации и загрузка исходных материалов в виде docx файла, на основе которых будет генерироваться новый структурированный конспект со всеми дополнительными заданиями.

Рисунок 5.11 – Подготовка к генерации материала

В результате генерации получается новый материал, который представлен на рисунке 5.12.

Рисунок 5.12 – Сгенерированный материал

Данный материал доступен для публикации и также для внесения правок, если таковые необходимы.

5.3.3 Публикация учебных материалов

Сгенерированный материал находится в статусе «Черновика», но, чтобы он стал доступен для работы студентам системы, его нужно опубликовать. Данное действие представлено на рисунке 5.13.

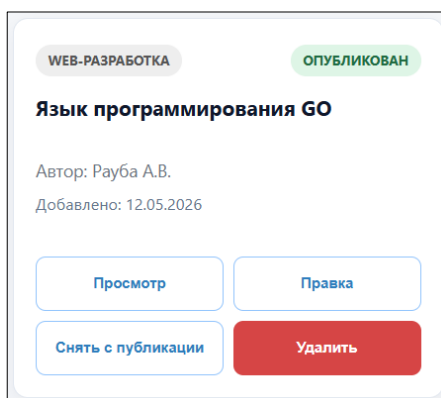


Рисунок 5.13 – Публикация материала

После выполнения данного действия, данный материал будет виден другим преподавателям и студентам данной системы.

5.4 Руководство администратора

5.4.1 Аутентификация

Если администратор не зашёл в аккаунт, то ему необходимо перейти на страницу авторизации, где он сможет войти, используя свой логин и пароль. Панель администратора после входа представлена на рисунке 5.14.

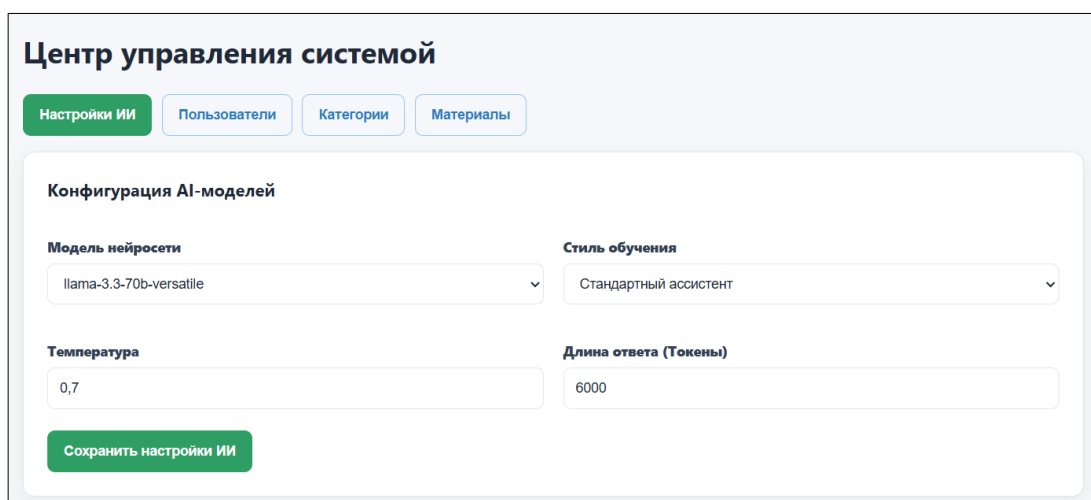


Рисунок 5.14 – Панель администратора

На данной панели доступно четыре вкладки с которыми администратор системы работает.

5.4.2 Конфигурация параметров AI-моделей

Одна из основных вкладок панели администратора – настройка ИИ. Данная вкладка представлена на рисунке 5.15.

The screenshot shows the 'Центр управления системой' (System Management Center) interface. The 'Настройки ИИ' (AI Settings) tab is selected. The 'Конфигурация AI-моделей' (AI Model Configuration) section contains the following fields:

- Модель нейросети** (Neural Network Model): A dropdown menu with 'llama-3.3-70b-versatile' selected.
- Стиль обучения** (Learning Style): A dropdown menu with 'Стандартный ассистент' (Standard Assistant) selected.
- Температура** (Temperature): A text input field with the value '0,7'.
- Длина ответа (Токены)** (Response Length (Tokens)): A text input field with the value '6000'.
- Сохранить настройки ИИ** (Save AI Settings): A green button at the bottom.

Рисунок 5.15 – Вкладка настроек ИИ

На данной вкладке администратор выбирает модель нейросети и другие ключевые параметры, на основе которых будет генерироваться учебный материал.

5.4.3 Управление учетными записями пользователей

Также пользователю доступна возможность управлять учетными записями пользователей. Данная вкладка представлена на рисунке 5.16.

The screenshot shows the 'Центр управления системой' (System Management Center) interface. The 'Пользователи' (Users) tab is selected. The 'Управление учетными записями' (Manage Accounts) section includes:

- A search bar: 'Введите ФИО или логин пользователя...' (Enter user name or login...).
- Filters: 'Все роли' (All roles) and 'ФИО (А-Я)' (Name A-Z).
- A table of users with columns: ФИО Пользователя, Логин, Роль, Изменить роль, and Действие.

ФИО Пользователя	Логин	Роль	Изменить роль	Действие
Александрович Иван Александрович	bezbashen@gmail.com	СТУДЕНТ	Студент	Заблокировать Удалить
Иванов Иван Иванович	test@gmail.com	СТУДЕНТ	Студент	Заблокировать Удалить
Мамонько Денис Александрович (Вы)	administrator@gmail.com	АДМИНИСТРАТОР	Администратор	Заблокировать Удалить
Рауба Арсений Владимирович	raubars@gmail.com	УЧИТЕЛЬ	Учитель	Заблокировать Удалить
Семёнов Даниил Вячеславович	bel4r@gmail.com	УЧИТЕЛЬ	Учитель	Заблокировать Удалить

Рисунок 5.16 – Вкладка пользователей

На данной вкладке отображена таблица пользователей системы и ряд функций для работы с пользователями. Помимо этого, администратор ответственен за управление тематическими категориями и материалами.

5.4.4 Управление профилем

Администратору, как и другим пользователям под другими ролями, доступна возможность управлять своим профилем. Окно профиля представлено на странице 3.17.

ФИО
Мамонько Денис Александрович

Роль
АДМИНИСТРАТОР

Текущий логин
administrator@gmail.com

Дата регистрации
05.05.2026

Изменить логин

Введите новый логин

Изменить логин

Безопасность

Текущий пароль
Введите текущий пароль

Новый пароль
Введите новый пароль

Подтверждение
Подтвердите новый пароль

Изменить пароль

Рисунок 3.17 – Окно профиля

На данной странице можно посмотреть информацию о профиле: дату регистрации, роль, ФИО и логин. Также доступна возможность изменить пароль и логин.

5.5 Выводы по разделу

1. В разделе приведено руководство пользователя, в котором пошагово описаны действия участников системы AI-Education с разными ролями: гость, обучающийся, преподаватель и администратор. Для каждой роли выделен свой набор функций, отражающий типичные сценарии работы с web-приложением и снижающий порог входа для новых пользователей.

2. Гости могут ознакомиться с описанием системы на главной странице, просматривать каталог опубликованных учебных материалов с ограниченным просмотром конспекта, работать с публичным фрагментом терминов и при необходимости зарегистрироваться или войти в систему для доступа к полному функционалу.

3. Обучающиеся после аутентификации получают доступ к учебной панели: просмотр опубликованных материалов с поиском, фильтрацией и сортировкой, открытие лекции, изучение конспекта и терминов, прохождение тестов, работа с вопросами для самопроверки и ситуационной задачей, экспорт материала в текстовый файл, а также ведение прогресса по материалу и просмотр мини-статистики. В профиле доступны просмотр данных и смена логина и пароля.

4. Преподаватели формируют учебный контент: задают параметры генерации (объём конспекта, число вопросов, категория), загружают исходный PDF/DOCX, запускают генерацию модуля с помощью ИИ, затем редактируют конспект, термины, тесты и практическое задание, при необходимости регенерируют отдельные вопросы теста, настраивают публичный доступ для гостей, публикуют или снимают с публикации материал, удаляют лишнее. Могут просматривать свои и чужие опубликованные материалы, проходить тест в режиме проверки и экспортировать материал. Управление личным профилем аналогично обучающемуся.

5. Администраторы обеспечивают работоспособность платформы: настройка параметров ИИ (модель, стиль/промт, температура, лимит токенов), ведение категорий знаний, управление пользователями (поиск, фильтрация, сортировка, смена роли, блокировка, удаление при отсутствии привязанных материалов), а также надзор за учебными материалами (поиск, фильтрация, сортировка, просмотр, снятие с публикации, удаление) без необходимости быть автором материала.

Заключение

1. В ходе работы над проектом разработано web-приложение «AI-Education», предназначенное для поддержки образовательного процесса: автоматизированного создания конспектов, терминов, тестов и практических заданий на основе загружаемых документов и последующей работы обучающихся с готовым материалом. В системе реализованы четыре роли: гость, обучающийся, преподаватель и администратор. Разграничение прав обеспечивает безопасный доступ к функциям:

- гость – к публичному каталогу и регистрации;
- обучающийся – к полному изучению опубликованных лекций и тестированию;
- преподаватель – к генерации и редактированию своих материалов и публикации;
- администратор – к настройке ИИ, пользователям, категориям и контролю материалов.

2. Функциональность охватывает полный цикл работы с учебным модулем: регистрация и аутентификация (JWT, хеширование пароля), загрузка PDF/DOCX, вызов внешнего LLM API, сохранение структурированного контента в базе, редактирование и публикация материалов, прохождение тестов и учёт прогресса обучающегося, экспорт материала, а также административные сценарии управления пользователями, категориями и конфигурацией нейросети. Интерфейс построен на отдельных панелях по ролям с единой навигацией и уведомлениями о результатах операций.

3. Реляционная база данных реализована в Microsoft SQL Server. В составе схемы используются таблицы ролей и пользователей, категорий знаний, учебных материалов, прогресса обучающихся по материалам и системных настроек ИИ (в скрипте создания БД – шесть основных таблиц, связанных внешними ключами). Доступ к данным из серверной части выполняется через драйвер mssql с параметризованными SQL-запросами.

4. Архитектура приложения клиент – сервер:

- серверная часть – Node.js и фреймворк Express (маршруты, контроллеры, middleware для JWT и загрузки файлов multer);
- клиентская часть – React (сборка Vite), react-router-dom, axios;
- для интеллектуальной генерации используется обращение к REST API нейросетевого провайдера по настраиваемому базовому URL и ключу.

Такое разделение упрощает развёртывание и дальнейшее развитие системы.

5. Проверка работоспособности основных сценариев оформлена в виде функционального тестирования.

Список используемых источников

1. Quizlet [Электронный ресурс] / Режим доступа: <https://quizlet.com/> – Дата доступа: 05.05.2026.
2. Stepik [Электронный ресурс] / Режим доступа: <https://stepik.org/> – Дата доступа: 05.05.2026.
3. Node.js Documentation [Электронный ресурс] / Режим доступа: <https://nodejs.org/docs/latest/api/> – Дата доступа: 26.04.2026.
4. Express.js 4.x API Reference [Электронный ресурс] / Режим доступа: <https://expressjs.com/en/4x/api.html> – Дата доступа: 26.04.2026.
5. React – документация [Электронный ресурс] / Режим доступа: <https://react.dev/> – Дата доступа: 28.04.2026.
6. Vite – руководство [Электронный ресурс] / Режим доступа: <https://vite.dev/guide/> – Дата доступа: 28.04.2026.
7. React Router – документация [Электронный ресурс] / Режим доступа: <https://reactrouter.com/> – Дата доступа: 28.04.2026.
8. Microsoft SQL Server – документация [Электронный ресурс] / Режим доступа: <https://learn.microsoft.com/sql/sql-server/> – Дата доступа: 26.04.2026.
9. node-mssql (пакет для Node.js) [Электронный ресурс] / Режим доступа: <https://www.npmjs.com/package/mssql> – Дата доступа: 26.04.2026.
10. jsonwebtoken [Электронный ресурс] / Режим доступа: <https://www.npmjs.com/package/jsonwebtoken> – Дата доступа: 27.04.2026.
11. bcryptjs [Электронный ресурс] / Режим доступа: <https://www.npmjs.com/package/bcryptjs> – Дата доступа: 27.04.2026.
12. multer [Электронный ресурс] / Режим доступа: <https://www.npmjs.com/package/multer> – Дата доступа: 26.04.2026.
13. Axios [Электронный ресурс] / Режим доступа: <https://axios-http.com/docs/intro> – Дата доступа: 27.04.2026.
14. Groq – документация API [Электронный ресурс] / Режим доступа: <https://console.groq.com/docs/overview> – Дата доступа: 28.04.2026.

Приложение А

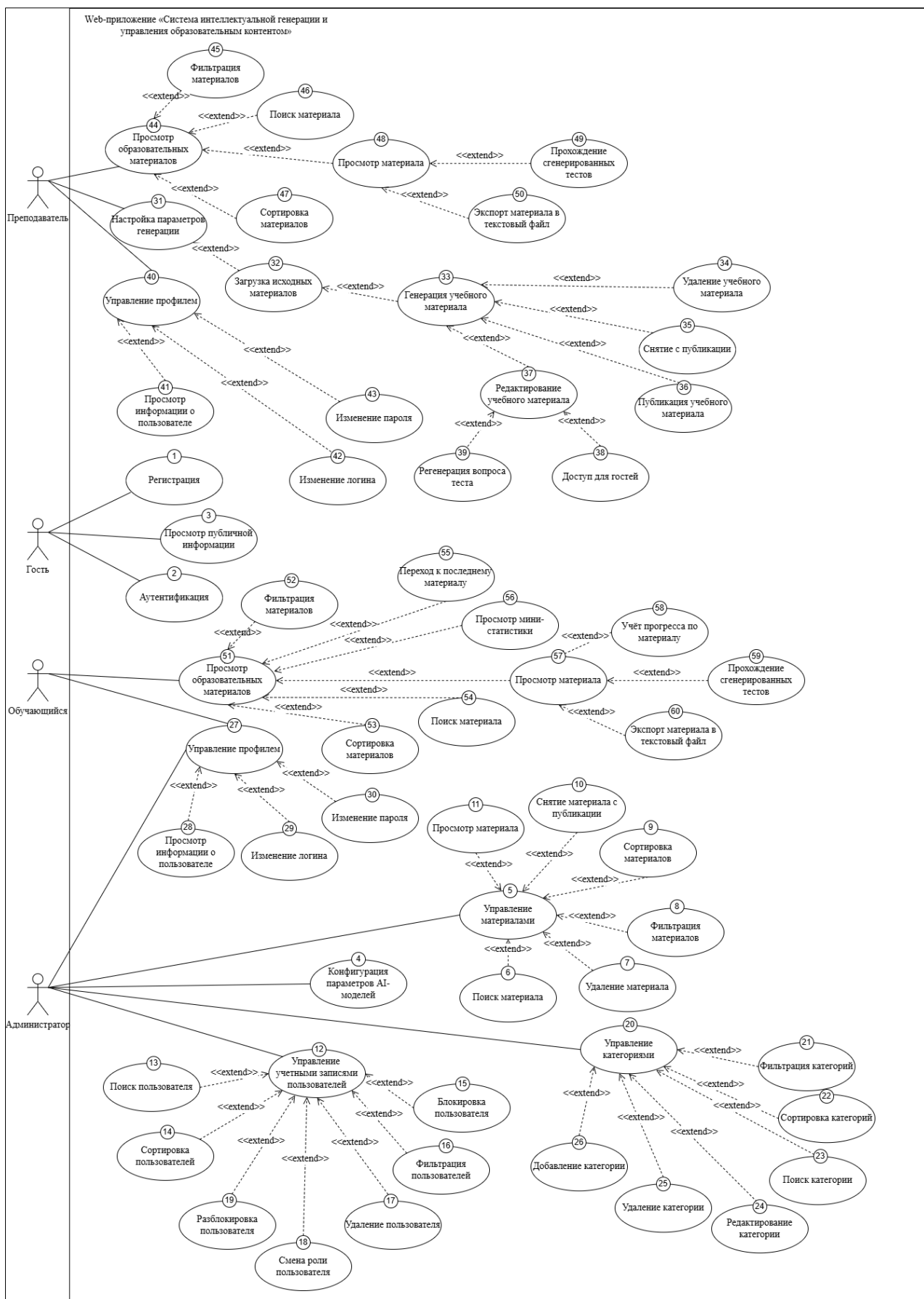


Рисунок – Диаграмма вариантов использования

Приложение Б

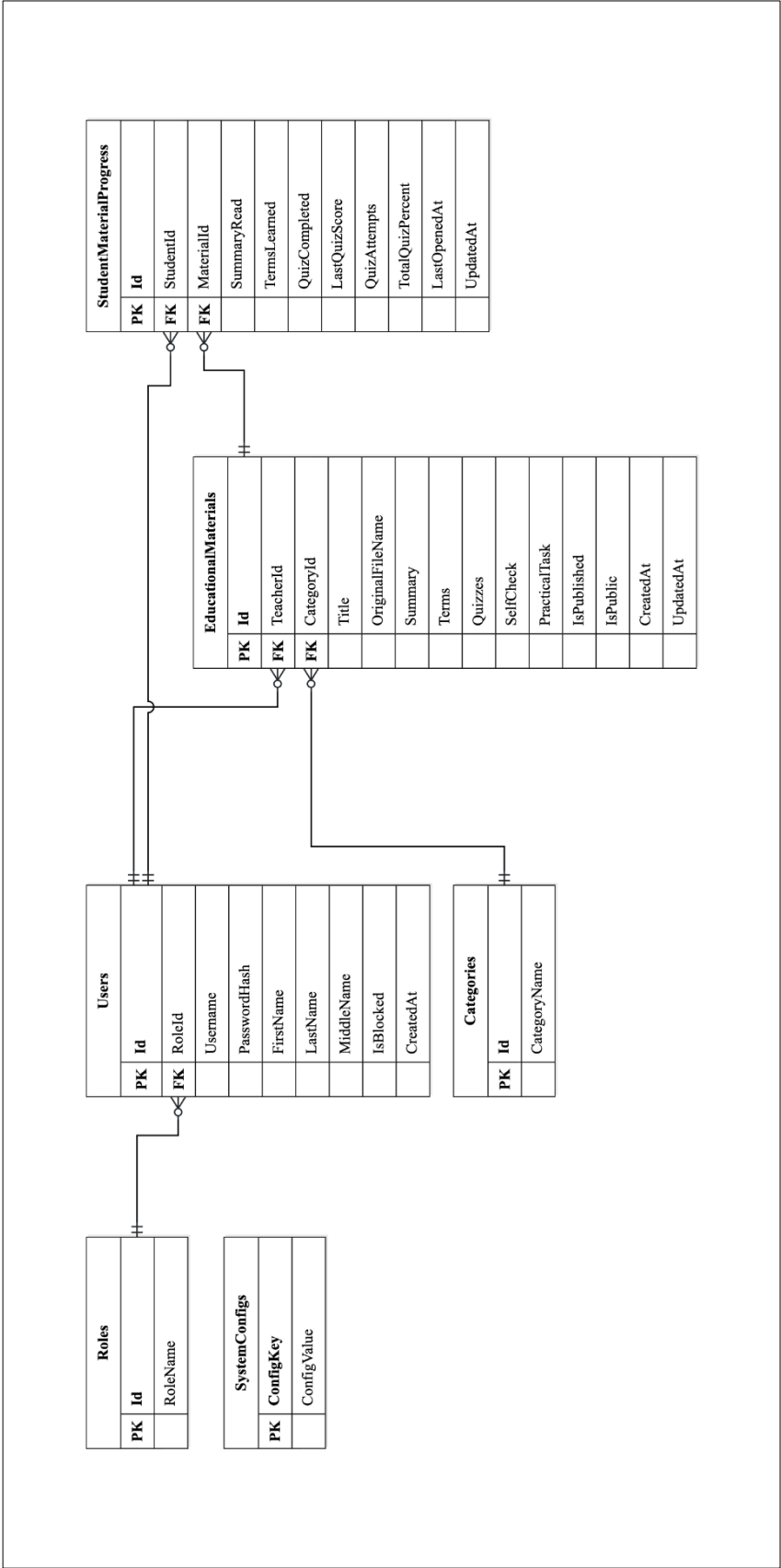


Рисунок – Логическая схема базы данных

Приложение В

```
const sql = require('mssql');
require('dotenv').config();
const dbConfig = {
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  server: process.env.DB_SERVER,
  database: process.env.DB_NAME,
  options: {
    encrypt: false,
    trustServerCertificate: true,
    enableArithAbort: true
  }
};
const connectDB = async () => {
  try {
    const pool = await sql.connect(dbConfig);
    console.log('Подключение к MSSQL установлено успешно');
    return pool;
  } catch (error) {
    console.error('Ошибка подключения к БД: ', error.message);
    process.exit(1);
  }
};
module.exports = { sql, connectDB };
```

Листинг – Подключение к Microsoft SQL Server

Приложение Г

```
exports.login = async (req, res) => {
  const { username, password } = req.body;
  try {
    const result = await sql.query`SELECT u.*, r.RoleName
FROM Users u JOIN Roles r ON u.RoleId = r.Id WHERE u.Username =
${username}`;
    const user = result.recordset[0];

    if (!user) return res.status(400).json({ message:
'Неверный логин или пароль' });
    if (user.IsBlocked) return res.status(403).json({
message: 'Ваш аккаунт заблокирован' });

    const isMatch = await bcrypt.compare(password,
user.PasswordHash);
    if (!isMatch) return res.status(400).json({ message:
'Неверный логин или пароль' });

    const token = jwt.sign({
      userId: user.Id,
      role: user.RoleName,
      username: user.Username,
      firstName: user.FirstName,
      lastName: user.LastName,
      middleName: user.MiddleName
    }, process.env.JWT_SECRET, { expiresIn: '24h' });

    res.json({
      token,
      role: user.RoleName,
      username: user.Username,
      firstName: user.FirstName,
      lastName: user.LastName,
      middleName: user.MiddleName
    });
  } catch (err) { res.status(500).json({ message: ' Ошибка
сервера' }); }
};
```

Листинг – Реализация метода login

Приложение Д

```

exports.getAllMaterials = async (req, res) => {
  try {
    let query;
    const baseQuery = `
      SELECT m.*, u.FirstName, u.LastName, u.MiddleName,
c.CategoryName
      FROM EducationalMaterials m
      JOIN Users u ON m.TeacherId = u.Id
      JOIN Categories c ON m.CategoryId = c.Id
    `;

    if (!req.user) {
      query = `${baseQuery} WHERE m.IsPublished = 1 AND
m.IsPublic = 1 ORDER BY m.CreatedAt DESC`;
    } else if (req.user.role === 'Student') {
      query = `${baseQuery} WHERE m.IsPublished = 1 ORDER
BY m.CreatedAt DESC`;
    } else if (req.user.role === 'Teacher') {
      query = `${baseQuery} WHERE m.TeacherId =
${req.user.userId} OR m.IsPublished = 1 ORDER BY m.CreatedAt
DESC`;
    } else {
      query = `${baseQuery} ORDER BY m.CreatedAt DESC`;
    }

    const result = await sql.query(query);
    res.json(result.recordset);
  } catch (err) { res.status(500).json({ message: 'Ошибка базы
данных' }); }
};

exports.getMaterialById = async (req, res) => {
  try {
    const result = await sql.query`
      SELECT m.*, u.FirstName, u.LastName, u.MiddleName,
c.CategoryName
      FROM EducationalMaterials m
      JOIN Users u ON m.TeacherId = u.Id
      JOIN Categories c ON m.CategoryId = c.Id
      WHERE m.Id = ${req.params.id}
    `;
    const m = result.recordset[0];
    if (!m) return res.status(404).json({ message: 'Материал
не найден' });

    m.Terms = JSON.parse(m.Terms || '[]');
    m.Quizzes = normalizeQuizzes(JSON.parse(m.Quizzes ||
'[]'));
    m.SelfCheck = JSON.parse(m.SelfCheck || '[]');
    m.PracticalTask = JSON.parse(m.PracticalTask || 'null');
    res.json(sanitizeMaterialContent(m));
  }
};

```

```
    } catch (err) { res.status(500).json({ message: 'Ошибка загрузки' }); }  
  };  
  
exports.getCategories = async (req, res) => {  
  try {  
    const result = await sql.query`SELECT * FROM Categories  
ORDER BY CategoryName`;  
    res.json(result.recordset);  
  } catch (err) { res.status(500).json({ message: 'Ошибка БД' }); }  
};
```

Листинг – Публичный доступ к каталогу и карточке материала

Приложение Е

```

const filtered = materials.filter(m => {
  const authorFullName = `${m.LastName} ${m.FirstName}
${m.MiddleName || ''}`.toLowerCase();
  const matchesSearch =
m.Title.toLowerCase().includes(search.toLowerCase()) ||
authorFullName.includes(search.toLowerCase());
  const matchesCat = filterCat === 'Bce' || m.CategoryName ===
filterCat;
  return matchesSearch && matchesCat;
});

const sorted = [...filtered].sort((a, b) => {
  switch (sortBy) {
    case 'created_desc': return new Date(b.CreatedAt) - new
Date(a.CreatedAt);
    case 'created_asc': return new Date(a.CreatedAt) - new
Date(b.CreatedAt);
    case 'updated_desc': return new Date(b.UpdatedAt) - new
Date(a.UpdatedAt);
    case 'updated_asc': return new Date(a.UpdatedAt) - new
Date(b.UpdatedAt);
    case 'title_asc': return a.Title.localeCompare(b.Title,
'ru');
    case 'title_desc': return b.Title.localeCompare(a.Title,
'ru');
    default: return 0;
  }
});

const openMaterial = async (materialId) => {
  const res = await api.get(`/content/${materialId}`);
  setSelected(res.data);
  updateProgress(res.data.Id, { markOpened: true });
};

```

Листинг – Просмотр и открытие материала